**Aulas 06 - Projeto de Aprendizagem de Máquina de Ponta a Ponta**

*Exemplo prático para Predição de preços de imóveis.*




#Setup

O projeto requer Python 3.7 ou acima:

In [1]:
```
import sys

assert sys.version_info >= (3, 7)
```

Também requer Scikit-Learn ≥ 1.0.1:

In [2]:
```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

# Obtendo os Dados

###Contexto de Negócio

*Bem vindo ao CDN-Fatec Votorantim! Sua atividade será prever o valor médio de casas nos Distritos da Califórnia, dado um número de atributos destes distritos.*

Este modelo será utilizado para auxiliar na localização de áreas com maior potencial para desenvolvimento de empreendimentos imobiliários.

Atualmente, a empresa conta com um grupo de corretores de imóveis, que são especialistas em avaliação de valores imobiliários. Apesar da expertise no mercado deste grupo de profissionais, em geral eles ainda tem uma taxa de acerto em torno de 70%.

O objetivo do modelo, é trazer uma ferramenta capaz de melhorar este desempenho, simplificando o processo de avaliação de preços, e dando mais tempo para que estes profissionais também possam se dedicar a outras atividades.

## Baixando os Dados

Este script faz o download do arquivo "housing.tgz", que será utilizado neste modelo!

*Esta é uma boa prática, pois fica simples atualizar os dados utilizados pelo modelo. Muito melhor do que fazer o download e link manualmente.*

In [29]:
```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
    with tarfile.open(tarball_path) as housing_tarball:
            housing_tarball.extractall(path="datasets")
    return pd.read_csv(Path("datasets/housing/housing.csv"))

# Gravando dados baixados em housing (objeto DataFrame do Pandas)
housing = load_housing_data()
```

```
/tmp/ipykernel_370/34910664.py:13: DeprecationWarning: Python
3.14 will, by default, filter extracted tar archives and reject
files or modify their metadata. Use the filter argument to
control this behavior.
  housing_tarball.extractall(path="datasets")
```

## Dando uma olhadinha na Estrutura dos Dados

In [30]:
```
#Lendo e exibindo os dados das 5 primeiras linhas
housing.head()
```

Out[30]:

| | longitude | latitude | housing_median_age | total_rooms | total_bedr |
|---|---|---|---|---|---|
| 0 | -122.23 | 37.88 | 41.0 | 880.0 | 129.0 |
| 1 | -122.22 | 37.86 | 21.0 | 7099.0 | 1106.0 |
| 2 | -122.24 | 37.85 | 52.0 | 1467.0 | 190.0 |
| 3 | -122.25 | 37.85 | 52.0 | 1274.0 | 235.0 |
| 4 | -122.25 | 37.85 | 52.0 | 1627.0 | 280.0 |

In [31]:
```
#Consultando um descritivo Geral dos Dados, Quantidade de Linhas, Quantidade de valores não nulos e o Tipo do dado
housing.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object 
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

In [32]:
```
#O campo ocean_proximity é um campo texto, provavelmente um ENUM, com este comando vamos ver quantos Enumeradores existem e quantos valores em cada opção
housing["ocean_proximity"].value_counts()
```

Out[32]:

| | count |
|---|---|
| **ocean_proximity** | |
| **<1H OCEAN** | 9136 |
| **INLAND** | 6551 |
| **NEAR OCEAN** | 2658 |
| **NEAR BAY** | 2290 |
| **ISLAND** | 5 |

**dtype:** int64

In [33]:
```
# Aqui temos uma visualização resumida de cada atributo numérico
# Os atributos count(quantidade), mean(média), min e max são auto-
explicativos
# std é o desvio padrão, que cálcula a dispersão de valores
# 25%, 50% e 75% indica que este número indica que o % indicado contém
todos os valores abaixo dele. Exemplo 25% das casas tem
"housing_median_age" menor ou igual a 18.

housing.describe()
```

Out[33]:

| | longitude | latitude | housing_median_age | total_roo |
|---|---|---|---|---|
| count | 20640.000000 | 20640.000000 | 20640.000000 | 20640.0000 |
| mean | -119.569704 | 35.631861 | 28.639486 | 2635.76308 |
| std | 2.003532 | 2.135952 | 12.585558 | 2181.61525 |
| min | -124.350000 | 32.540000 | 1.000000 | 2.000000 |
| 25% | -121.800000 | 33.930000 | 18.000000 | 1447.75000 |
| 50% | -118.490000 | 34.260000 | 29.000000 | 2127.00000 |
| 75% | -118.010000 | 37.710000 | 37.000000 | 3148.00000 |
| max | -114.310000 | 41.950000 | 52.000000 | 39320.0000 |

O Bloco seguinte, define a função save_fig, que gravará as imagens geradas para os gráficos na pasta `images/end_to_end_project` . Assim todas as imagens plotadas abaixo, ficam disponíveis para download.

In [34]:
```
# Código para salvar as imagens em alta resolução

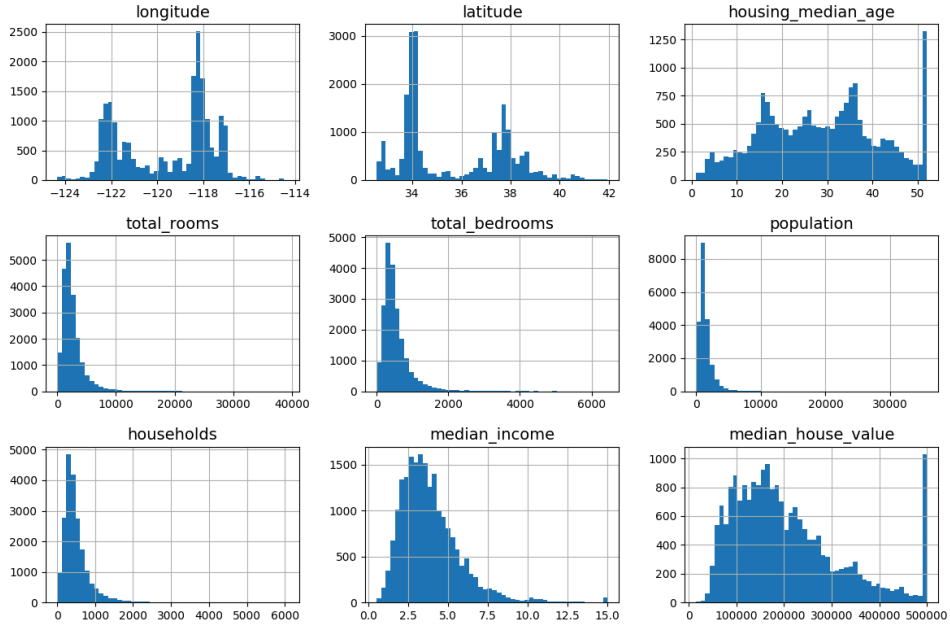
IMAGES_PATH = Path() / "images" / "end_to_end_project"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png",
resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

In [35]:
```
import matplotlib.pyplot as plt

# As próximas 5 linhas definem as fontes dos gráficos, apenas para
estética
plt.rc('font', size=14)
plt.rc('axes', labelsize=14, titlesize=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsize=10)
plt.rc('ytick', labelsize=10)

# comando para criar histogramas a partir dos dados
housing.hist(bins=50, figsize=(12, 8))
save_fig("attribute_histogram_plots")  # gravando as figuras
# Exibindo os gráficos no console
plt.show()
```




**Observações importantes**

**- median_income** - O atributo median_income não está em dólares. Ele está em dezenas de milhares de dólares, ou seja, uma linha com o valor 3, representa que a renda é 30 mil dólares. Isso não é um problema para o trabalho, mas é importante conhecer esses detalhes.

**- median_house_value e housing_median_age** - Estes campos foram limitados. Todos os valores acima de um determinado valor, estão com o mesmo valor. Isso é problemático, principalmente para o median_house_value, que é uma informação chave para o que queremos obter.

**- inclinação dos histogramas** - É possível observar na maioria dos histogramas, que a média dos dados está inclinada mais para a direita. Isso pode dificultar o processo de aprendizado de alguns algoritmos.

## Criando um conjunto de testes

Aqui será separado uma parte dos dados, que serão utilizados para os testes do modelo. Em geral, algo em torno de 20% dos dados.

In [36]:
```
import numpy as np

def shuffle_and_split_data(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

In [37]:
```
train_set, test_set = shuffle_and_split_data(housing, 0.2)
len(train_set)
```

Out[37]: 16512

In [38]:
```
len(test_set)
```

Out[38]: 4128

A definição da seed (semente) 42, é para garantir que o mesmo conjunto de dados será selecionado dos dados, para o conjunto de testes.

In [39]:
```
np.random.seed(42)
```

Infelizmente, isso não garante totalente as mesmas saídas em todas as execuções. Podem ocorrer alterações nos dados de entrada, nas bibliotecas utilizadas e várias outras fontes de variações nos dados.

Para este exemplo, isso é suficiente, mas dependendo do projeto, é necessário outras atitudes.

Nota: Outra fonte de aleatoriedade, é baseada na função `hash()` do Python. Para evitar este problema, defina a variável de ambiente `PYTHONHASHSEED` para `"0"` _antes_ do Python inicializar.

Felizmente, rodando no Google Colab, a variável já está definida para você.

In [40]:
```
from zlib import crc32

def is_id_in_test_set(identifier, test_ratio):
    return crc32(np.int64(identifier)) < test_ratio * 2**32

def split_data_with_id_hash(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_,
test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]
```

In [41]:
```
# Como os imóveis não tem um identificador, será necessário adicionar
uma coluna `index` aos dados
housing_with_id = housing.reset_index()
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2,
"index")
```

In [42]:
```
housing_with_id["id"] = housing["longitude"] * 1000 +
housing["latitude"]
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2,
"id")
```

In [43]:
```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing, test_size=0.2,
random_state=42)
```

In [44]:
```
test_set["total_bedrooms"].isnull().sum()
```

Out[44]: np.int64(44)

**Estratificação de Resultados**

O método acima, cria uma amostra de dados aleatória, e em geral, isso é bom. Porém, há situações, em que estatísticamente, há diferença entre a quantidade de dados em cada categoria, e o ideal, é que a sua amostra de dados, seja o mais representativa possível da realidade.

Exemplo:

Segundo o Censo 2022, a população brasileira possui 51,48% de mulheres e 48,52% de Homens. Uma empresa de pesquisas (IBOP) precisa de uma amostra, que fique o mais próxima possível desta representatividade entre homens e mulheres. Não pode ser totalmente aleatório.

A preparação de uma amostra, buscando manter a proporção de itens entre cada categoria, se chama **Amostragem Estratificada**.

*Avaliação de probabilidade de se obter uma amostra que não corresponte a realidade*

Para encontrar a probabilidade de que uma amostra aleatória de 1.000 pessoas contenha menos de 48,5% de mulheres ou mais de 53,5% de mulheres, quando a proporção de mulheres na população é de 51,5%, utilizamos a distribuição binomial [binomial distribution](https://en.wikipedia.org/wiki/Binomial_distribution).

O método `cdf()` da distribuição binomial nos fornece a probabilidade de que o número de mulheres seja igual ou menor do que um determinado valor.

In [45]:
```
#Como calcular os 12.4% de probabilidade de se obter uma amostra ruim

from scipy.stats import binom

sample_size = 1000
ratio_female = 0.515
proba_too_small = binom(sample_size, ratio_female).cdf(485 - 1)
proba_too_large = 1 - binom(sample_size, ratio_female).cdf(535)
print(proba_too_small + proba_too_large)
```

```
0.12408391499150825
```

Se preferir uma simulação, aqui está como você pode chegar aproximadamente ao mesmo resultado:

In [46]:
```
# Outra maneira de calcular a probabilidade de uma amostra ruim

np.random.seed(42)

samples = (np.random.rand(100_000, sample_size) <
ratio_female).sum(axis=1)
((samples < 485) | (samples > 535)).mean()
```

Out[46]: np.float64(0.12427)

**Estratificação por "median_income"**

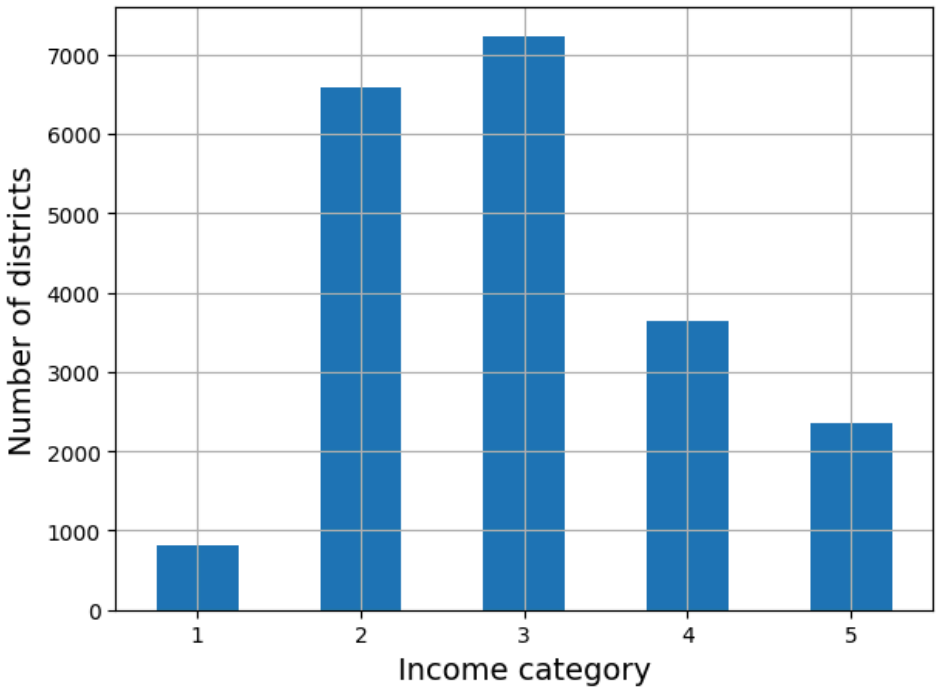
Os especialistas informaram que a renda média é um atributo muito importante para estimar os preços médios dos imóveis, portanto, vaos fazer uma estratificação a partir desta informação.

Como é um campo numérico, o primeiro passo será categorizar o atributo.

A função `cut()` faz a categorização dos valores, de acordo com os valores na lista "bins", criando com os rótulos em "labels".

In [47]:
```
housing["income_cat"] = pd.cut(housing["median_income"],
                               bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                               labels=[1, 2, 3, 4, 5])
```

In [48]:
```
housing["income_cat"].value_counts().sort_index().plot.bar(rot=0,
grid=True)
plt.xlabel("Income category")
plt.ylabel("Number of districts")
save_fig("housing_income_cat_bar_plot")  # extra code
plt.show()
```



Aqui serão gerados 10 divisões estratificadas diferentes do nosso conjunto de dados.

In [49]:
```
from sklearn.model_selection import StratifiedShuffleSplit

splitter = StratifiedShuffleSplit(n_splits=10, test_size=0.2,
random_state=42)
strat_splits = []
for train_index, test_index in splitter.split(housing,
housing["income_cat"]):
    strat_train_set_n = housing.iloc[train_index]
    strat_test_set_n = housing.iloc[test_index]
    strat_splits.append([strat_train_set_n, strat_test_set_n])
```

In [50]:
```
# Retornando a primeira divisão estratificada
strat_train_set, strat_test_set = strat_splits[0]
```

Esta é uma forma mais curta de se obter uma únida divisão estratificada diretamente dos dados:

In [51]:
```
strat_train_set, strat_test_set = train_test_split(
    housing, test_size=0.2, stratify=housing["income_cat"],
random_state=42)
```

In [52]:
```
strat_test_set["income_cat"].value_counts() / len(strat_test_set)
```

Out[52]:

| | count |
|---|---|
| **income_cat** | |
| **3** | 0.350533 |
| **2** | 0.318798 |
| **4** | 0.176357 |
| **5** | 0.114341 |
| **1** | 0.039971 |

**dtype:** float64

In [53]:
```
## Calculando os dados para comparar os percentuais de cada categoria
para
# - Amostra randômica,
# - Amostra estratificada
# - Dados globais

def income_cat_proportions(data):
    return data["income_cat"].value_counts() / len(data)

train_set, test_set = train_test_split(housing, test_size=0.2,
random_state=42)

compare_props = pd.DataFrame({
    "Overall %": income_cat_proportions(housing),
    "Stratified %": income_cat_proportions(strat_test_set),
    "Random %": income_cat_proportions(test_set),
}).sort_index()
compare_props.index.name = "Income Category"
compare_props["Strat. Error %"] = (compare_props["Stratified %"] /
                                   compare_props["Overall %"] - 1)
compare_props["Rand. Error %"] = (compare_props["Random %"] /
                                  compare_props["Overall %"] - 1)
(compare_props * 100).round(2)
```

Out[53]:

| Income Category | Overall % | Stratified % | Random % | Strat. Error % | Rand. Error % |
|---|---|---|---|---|---|
| 1 | 3.98 | 4.00 | 4.24 | 0.36 | 6.45 |
| 2 | 31.88 | 31.88 | 30.74 | -0.02 | -3.59 |
| 3 | 35.06 | 35.05 | 34.52 | -0.01 | -1.53 |
| 4 | 17.63 | 17.64 | 18.41 | 0.03 | 4.42 |
| 5 | 11.44 | 11.43 | 12.09 | -0.08 | 5.63 |

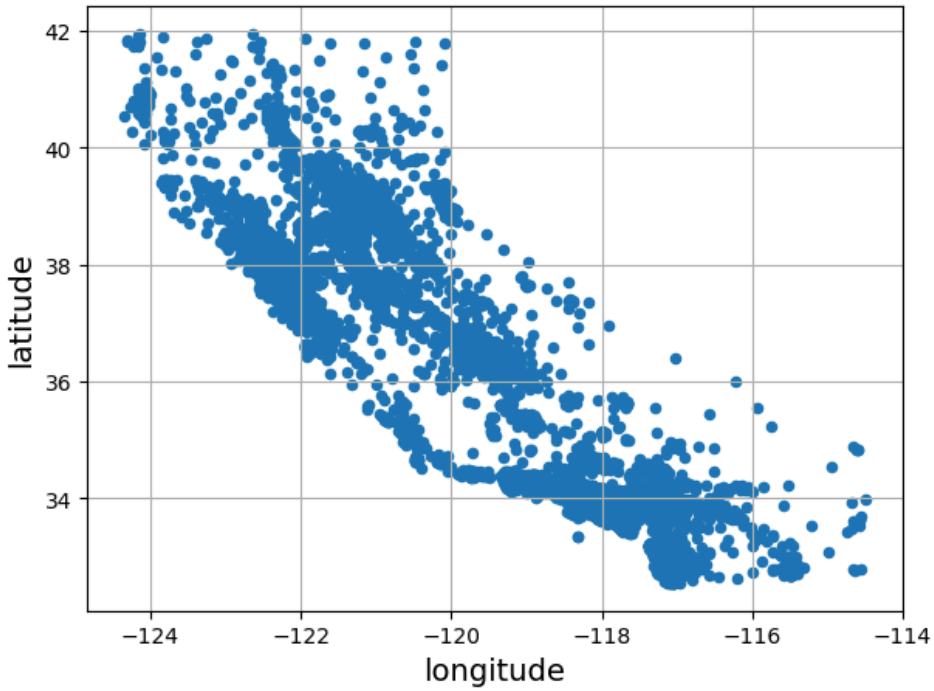
In [54]:
```
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

# Explore e Visualize os Dados para ter Insights

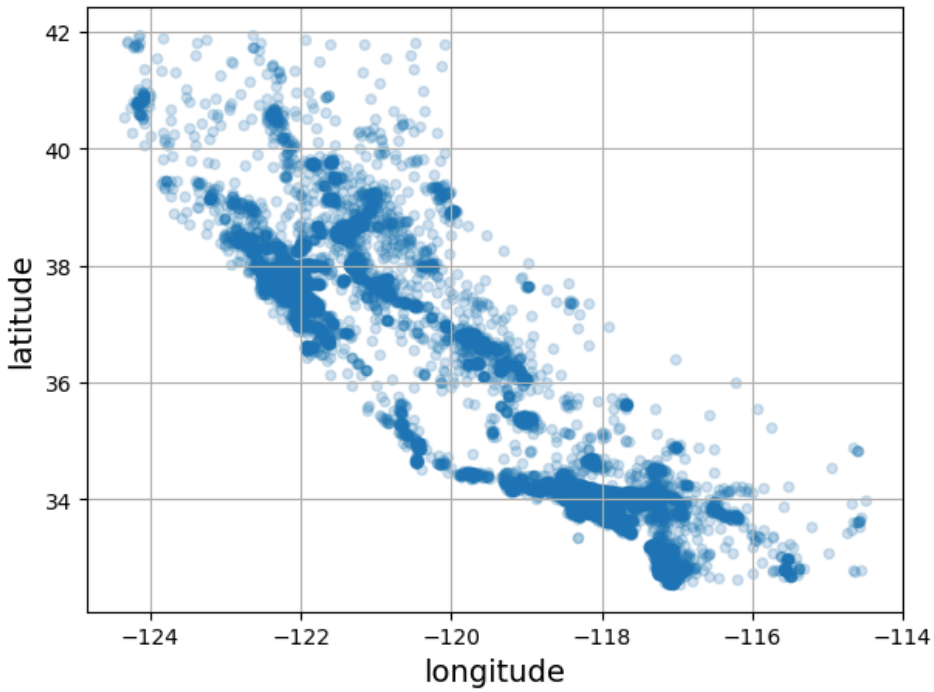
In [55]:
```
housing = strat_train_set.copy()
```

# Visualizando Dados Geográficos

In [56]:
```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)
save_fig("bad_visualization_plot")
plt.show()
```

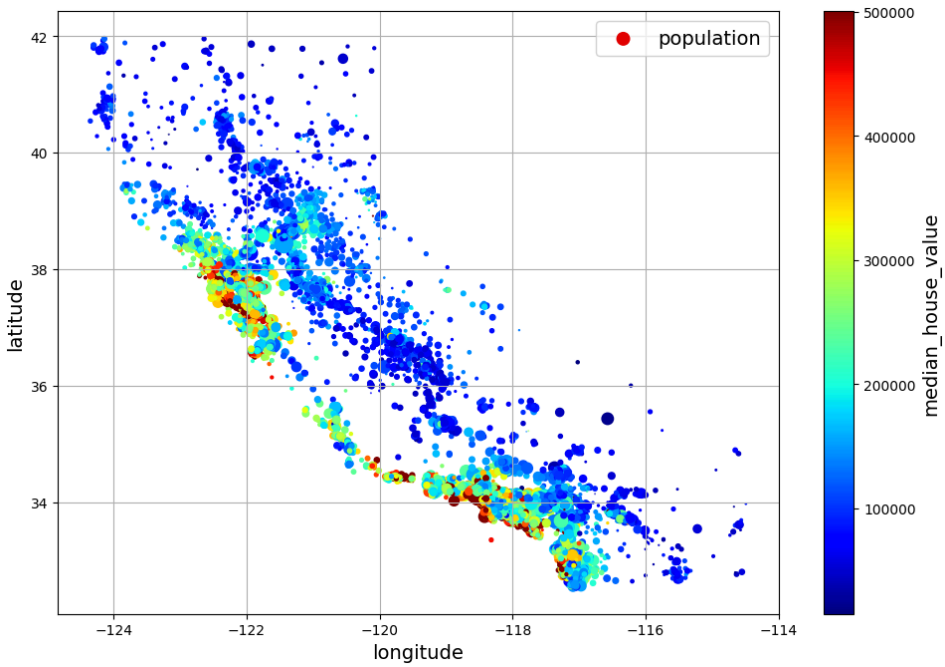


In [57]:
```
# Visualização melhor com transparência nos pontos,
# ajuda a visualizar locais com maior densidade
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
alpha=0.2)
save_fig("better_visualization_plot")
plt.show()
```



In [58]:
```
#Visualização diferenciando por população (tamanho do círculo) e valor
médio da casa (cor do círculo)

housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
             s=housing["population"] / 100, label="population",
             c="median_house_value", cmap="jet", colorbar=True,
             legend=True, sharex=False, figsize=(10, 7))
save_fig("housing_prices_scatterplot")  # extra code
plt.show()
```




O argumento `sharex=False` corrige um bug de visualização: sem ele, o eixo-x não exibe seus valores e labels (see: https://github.com/pandas-dev/pandas/issues/10611).

O código abaixo, gera a mesma imagem, mas com uma imagem do mapa da Califórnia ao fundo.

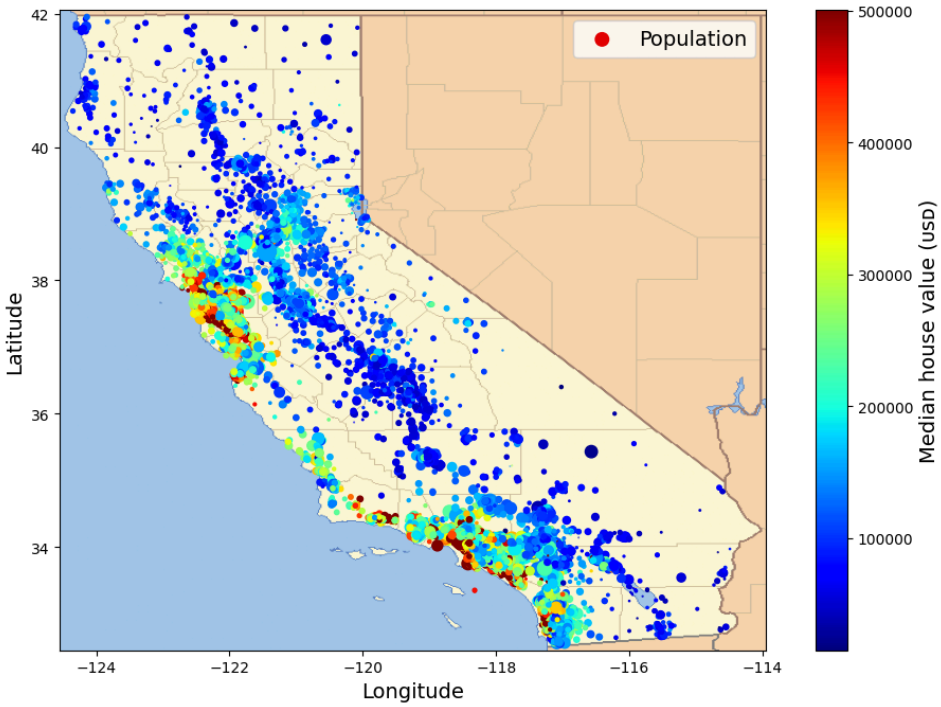
In [59]:
```
# Download the California image
filename = "california.png"
if not (IMAGES_PATH / filename).is_file():
    homl3_root = "https://github.com/ageron/handson-ml3/raw/main/"
    url = homl3_root + "images/end_to_end_project/" + filename
    print("Downloading", filename)
    urllib.request.urlretrieve(url, IMAGES_PATH / filename)

housing_renamed = housing.rename(columns={
    "latitude": "Latitude", "longitude": "Longitude",
    "population": "Population",
    "median_house_value": "Median house value (usd)"})
housing_renamed.plot(
             kind="scatter", x="Longitude", y="Latitude",
             s=housing_renamed["Population"] / 100, label="Population",
             c="Median house value (usd)", cmap="jet", colorbar=True,
             legend=True, sharex=False, figsize=(10, 7))

california_img = plt.imread(IMAGES_PATH / filename)
axis = -124.55, -113.95, 32.45, 42.05
plt.axis(axis)
plt.imshow(california_img, extent=axis)

save_fig("california_housing_prices_plot")
plt.show()
```

```
Downloading california.png
```



## Procurando Correlações

Nota: desde o Pandas 2.0.0, o defaults do argumento `numeric_only` é `False`, então precisamos definir explicitamente para evitar erros.

In [60]:
```
corr_matrix = housing.corr(numeric_only=True)
```

In [61]:
```
corr_matrix["median_house_value"].sort_values(ascending=False)
```

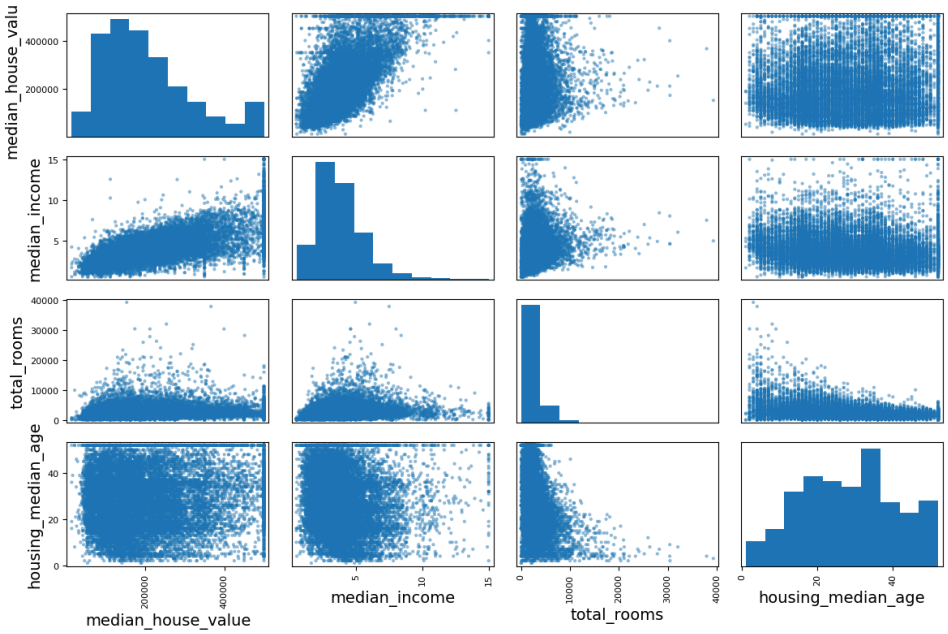
Out[61]:

| | median_house_value |
|---|---|
| **median_house_value** | 1.000000 |
| **median_income** | 0.688380 |
| **total_rooms** | 0.137455 |
| **housing_median_age** | 0.102175 |
| **households** | 0.071426 |
| **total_bedrooms** | 0.054635 |
| **population** | -0.020153 |
| **longitude** | -0.050859 |
| **latitude** | -0.139584 |

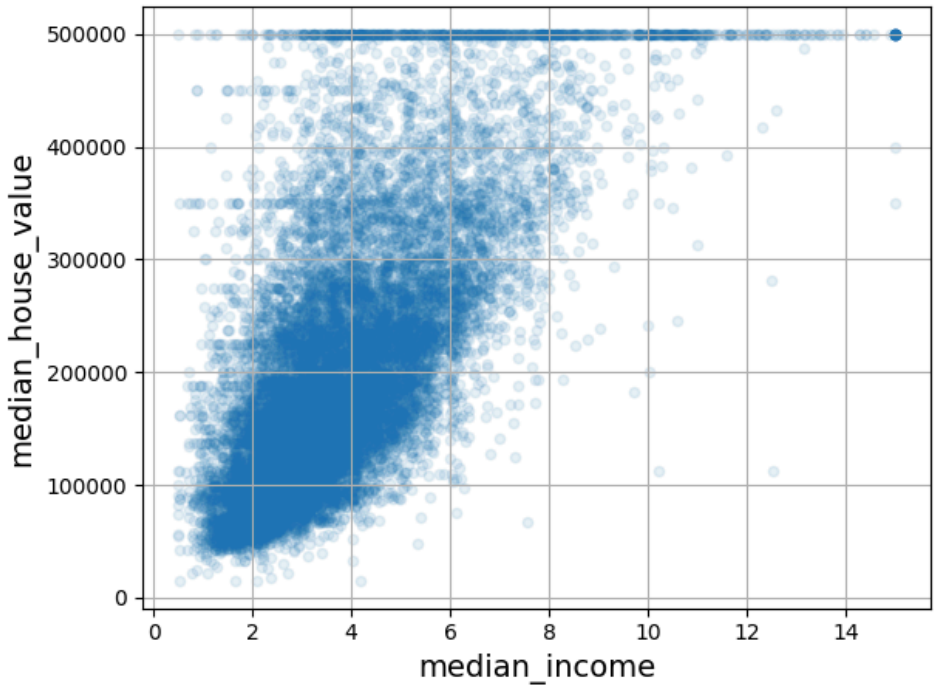
**dtype:** float64

In [62]:
```
from pandas.plotting import scatter_matrix

attributes = ["median_house_value", "median_income", "total_rooms",
              "housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
save_fig("scatter_matrix_plot")  # extra code
plt.show()
```




In [63]:
```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
             alpha=0.1, grid=True)
save_fig("income_vs_house_value_scatterplot")  # extra code
plt.show()
```



# Experimentando combinações de Atributos

Criando novos atributos:

- Cômodos por Casa `rooms_per_house`
- Proporção de Quartos `bedrooms_ratio`
- Pessoas por Casa `people_per_house`

In [66]:
```
housing["rooms_per_house"] = housing["total_rooms"] /
housing["households"]
housing["bedrooms_ratio"] = housing["total_bedrooms"] /
housing["total_rooms"]
housing["people_per_house"] = housing["population"] /
housing["households"]
```

Calculando correlações dos novos atributos contra o Preço Médio de Casa

In [67]:
```
corr_matrix = housing.corr(numeric_only=True)
corr_matrix["median_house_value"].sort_values(ascending=False)
```

Out[67]:

| | median_house_value |
|---|---|
| **median_house_value** | 1.000000 |
| **median_income** | 0.688380 |
| **rooms_per_house** | 0.143663 |
| **total_rooms** | 0.137455 |
| **housing_median_age** | 0.102175 |
| **households** | 0.071426 |
| **total_bedrooms** | 0.054635 |
| **population** | -0.020153 |
| **people_per_house** | -0.038224 |
| **longitude** | -0.050859 |
| **latitude** | -0.139584 |
| **bedrooms_ratio** | -0.256397 |

**dtype:** float64

# Preparando dados para algoritmo de Machine Learning

Serão utilizadas uma série de funções de código para preparar os dados para o processo de ML. Estas funções aos poucos, se tornaram uma biblioteca a ser utilizada em projetos futuros.

Primeiro, vamos reverter ao conjunto de treinamento original e separar a variável alvo (observe que `strat_train_set.drop()` cria uma cópia de `strat_train_set` sem a coluna, ele não modifica o próprio `strat_train_set` , a menos que você passe o parâmetro inplace=True):

In [68]:
```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

### Limpeza dos Dados

A maioria dos algoritmos de ML não lida bem com dados ausentes, portanto precisaremos cuidar disso.

Há alguns registros sem valor para o atributo `total_bedrooms`, temos 3 opções para resolver isso:
1. Abrir mão das regiões correspondentes
2. Abrir mão do atributo inteiro
3. Definir valores para algum valor (zero, média, mediana, etc...), fazendo uma imputação

Para isto, temos as seguintes funções:

```python
# opção 1

housing.dropna(subset=["total_bedrooms"], inplace=True)

# opção 2

housing.drop("total_bedrooms", axis=1)

# opção 3

median = housing["total_bedrooms"].median()
housing["total_bedrooms"].fillna(median, inplace=True)
```

Nos 3 casos, será criada uma cópia de `housing` e trabalharemos nessa cópia para evitar modificar (quebrar) o objeto `housing`. Também mostraremos o resultado de cada opção, mas filtrando apenas as linhas que originalmente continham um valor NaN.

In [69]:
```
null_rows_idx = housing.isnull().any(axis=1)
housing.loc[null_rows_idx].head()
```

Out[69]:

| | longitude | latitude | housing_median_age | total_rooms | total_ |
|---|---|---|---|---|---|
| **14452** | -120.67 | 40.50 | 15.0 | 5343.0 | NaN |
| **18217** | -117.96 | 34.03 | 35.0 | 2093.0 | NaN |
| **11889** | -118.05 | 34.04 | 33.0 | 1348.0 | NaN |
| **20325** | -118.88 | 34.17 | 15.0 | 4260.0 | NaN |
| **14360** | -117.87 | 33.62 | 8.0 | 1266.0 | NaN |

In [70]:
```
# Código para Opção 1
housing_option1 = housing.copy()

housing_option1.dropna(subset=["total_bedrooms"], inplace=True)

housing_option1.loc[null_rows_idx].head()
```

Out[70]:

| | longitude | latitude | housing_median_age | total_rooms | total_bedro |
|---|---|---|---|---|---|

In [71]:
```
# Código para Opção 2
housing_option2 = housing.copy()

housing_option2.drop("total_bedrooms", axis=1, inplace=True)

housing_option2.loc[null_rows_idx].head()
```

Out[71]:

| | longitude | latitude | housing_median_age | total_rooms | popu |
|---|---|---|---|---|---|
| **14452** | -120.67 | 40.50 | 15.0 | 5343.0 | 2503. |
| **18217** | -117.96 | 34.03 | 35.0 | 2093.0 | 1755. |
| **11889** | -118.05 | 34.04 | 33.0 | 1348.0 | 1098. |
| **20325** | -118.88 | 34.17 | 15.0 | 4260.0 | 1701. |
| **14360** | -117.87 | 33.62 | 8.0 | 1266.0 | 375.0 |

In [72]:
```
# Código para Opção 3

housing_option3 = housing.copy()

median = housing["total_bedrooms"].median()
housing_option3["total_bedrooms"].fillna(median, inplace=True)

housing_option3.loc[null_rows_idx].head()
```

```
/tmp/ipykernel_370/2644910045.py:6: FutureWarning: A value is
trying to be set on a copy of a DataFrame or Series through
chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method
will never work because the intermediate object on which we are
setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)',
try using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace
on the original object.


  housing_option3["total_bedrooms"].fillna(median,
inplace=True)
```

Out[72]:

| | longitude | latitude | housing_median_age | total_rooms | total_ |
|---|---|---|---|---|---|
| **14452** | -120.67 | 40.50 | 15.0 | 5343.0 | 434.0 |
| **18217** | -117.96 | 34.03 | 35.0 | 2093.0 | 434.0 |
| **11889** | -118.05 | 34.04 | 33.0 | 1348.0 | 434.0 |
| **20325** | -118.88 | 34.17 | 15.0 | 4260.0 | 434.0 |
| **14360** | -117.87 | 33.62 | 8.0 | 1266.0 | 434.0 |

## Opção 3 melhorada

O skicit-learn prove a classe SimpleImputer. Esta classe permite calcular e armazenar a média de todos os atributos, permitindo que os dados sejam imputados posteriormente em qualquer campo numérico.

In [73]:
```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

Separando os atributos numéricos para utilizar a estratégia de `"média"` (já que ela não pode ser calculada sobre atributos de texto, como ocean_proximity).

Os dados das médias ficam armazenados em `imputer.statistics` .

In [74]:
```
housing_num = housing.select_dtypes(include=[np.number])
```

In [75]:
```
imputer.fit(housing_num)
```

Out[75]:
```
SimpleImputer
SimpleImputer(strategy='median')
```

In [76]:
```
imputer.statistics_
```

Out[76]:
```
array([-118.51  ,   34.26  ,   29.    , 2125.    ,  434.    ,
1167.    ,
        408.    ,    3.5385])
```

Conferindo que os dados são os mesmos que calculados individualmente para cada atributo:

In [77]:
```
housing_num.median().values
```

Out[77]:
```
array([-118.51  ,   34.26  ,   29.    , 2125.    ,  434.    ,
1167.    ,
        408.    ,    3.5385])
```

Transformando o conjunto de dados de treinamento (training set):

In [78]:
```
X = imputer.transform(housing_num)
```

In [79]:
```
imputer.feature_names_in_
```

Out[79]:
```
array(['longitude', 'latitude', 'housing_median_age',
'total_rooms',
       'total_bedrooms', 'population', 'households',
'median_income'],
      dtype=object)
```

In [80]:
```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

In [81]:
```
housing_tr.loc[null_rows_idx].head()
```

Out[81]:

| | longitude | latitude | housing_median_age | total_rooms | total_ |
|---|---|---|---|---|---|
| **14452** | -120.67 | 40.50 | 15.0 | 5343.0 | 434.0 |
| **18217** | -117.96 | 34.03 | 35.0 | 2093.0 | 434.0 |
| **11889** | -118.05 | 34.04 | 33.0 | 1348.0 | 434.0 |
| **20325** | -118.88 | 34.17 | 15.0 | 4260.0 | 434.0 |
| **14360** | -117.87 | 33.62 | 8.0 | 1266.0 | 434.0 |

In [82]:
```
imputer.strategy
```

Out[82]: 'median'

In [83]:
```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

In [84]:
```
housing_tr.loc[null_rows_idx].head()  # not shown in the book
```

Out[84]:

| | longitude | latitude | housing_median_age | total_rooms | total_ |
|---|---|---|---|---|---|
| **14452** | -120.67 | 40.50 | 15.0 | 5343.0 | 434.0 |
| **18217** | -117.96 | 34.03 | 35.0 | 2093.0 | 434.0 |
| **11889** | -118.05 | 34.04 | 33.0 | 1348.0 | 434.0 |
| **20325** | -118.88 | 34.17 | 15.0 | 4260.0 | 434.0 |
| **14360** | -117.87 | 33.62 | 8.0 | 1266.0 | 434.0 |

In [ ]:
```
#from sklearn import set_config
#
# set_config(transform_output="pandas")  # scikit-learn >= 1.2
```

Agora vamos remover alguns outliers:

In [85]:
```
from sklearn.ensemble import IsolationForest

isolation_forest = IsolationForest(random_state=42)
outlier_pred = isolation_forest.fit_predict(X)
```

In [86]:
```
outlier_pred
```

Out[86]: array([-1, 1, 1, ..., 1, 1, 1])

Se você quisesse remover outliers, deveria execut o seguinte código:

In [87]:
```
#housing = housing.iloc[outlier_pred == 1]
#housing_labels = housing_labels.iloc[outlier_pred == 1]
```

## Manipulando Texto e Atributos de Categoria

Agora vamos fazer o pré-processamento do atributo categórico `ocean_proximity` :

In [88]:
```
housing_cat = housing[["ocean_proximity"]]
housing_cat.head(8)
```

Out[88]:

| | ocean_proximity |
|---|---|
| **13096** | NEAR BAY |
| **14973** | <1H OCEAN |
| **3785** | INLAND |
| **14689** | INLAND |
| **20507** | NEAR OCEAN |
| **1286** | INLAND |
| **18078** | <1H OCEAN |
| **4396** | NEAR BAY |

In [89]:
```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder()
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
```

In [90]:
```
housing_cat_encoded[:8]
```

Out[90]:
```
array([[3.],
       [0.],
       [1.],
       [1.],
       [4.],
       [1.],
       [0.],
       [3.]])
```

In [91]:
```
ordinal_encoder.categories_
```

Out[91]:
```
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR
OCEAN'],
       dtype=object)]
```

Um dos problemas desta representação, é que diversos algoritmos de ML, assumem que dois valores próximos são mais semelhantes que dois valores distantes. No caso de `ocean_proximiy, as categorias 0("<1H OCEAN") e 4("NEAR OCEAN") são claramente mais semelhantes que as catetorias 0("<1H OCEAN") e 1("INLAND").

Para resolver este problema, o Scikit-Learn fornece a classe OneHotEncoder, que criará colunas binárias para cada categoria em `ocean_proximity`.

In [92]:
```
from sklearn.preprocessing import OneHotEncoder

cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

In [93]:
```
housing_cat_1hot
```

Out[93]:
```
<Compressed Sparse Row sparse matrix of dtype 'float64'
	with 16512 stored elements and shape (16512, 5)>
```

Por padrão, a classe `OneHotEncoder` retorna uma matriz esparsa, mas podemos convertê-la em uma matriz densa, se necessário, chamando o método `toarray()`:

In [94]:
```
housing_cat_1hot.toarray()
```

Out[94]:
```
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

Alternativamente, você pode definir `sparse_output=False` ao criar o `OneHotEncoder` (observação: o hiperparâmetro `sparse` foi renomeado para `sparse_output` no Scikit-Learn 1.2):

In [95]:
```
cat_encoder = OneHotEncoder(sparse_output=False)
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
housing_cat_1hot
```

Out[95]:
```
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

In [96]:
```
cat_encoder.categories_
```

Out[96]:
```
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR
OCEAN'],
       dtype=object)]
```

In [97]:
```
df_test = pd.DataFrame({"ocean_proximity": ["INLAND", "NEAR BAY"]})
pd.get_dummies(df_test)
```

Out[97]:

| | ocean_proximity_INLAND | ocean_proximity_NEAR BAY |
|---|---|---|
| 0 | True | False |
| 1 | False | True |

In [98]:
```
cat_encoder.transform(df_test)
```

Out[98]:
```
array([[0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

In [99]:
```
df_test_unknown = pd.DataFrame({"ocean_proximity": ["<2H OCEAN",
"ISLAND"]})
pd.get_dummies(df_test_unknown)
```

Out[99]:

| | ocean_proximity_<2H OCEAN | ocean_proximity_ISLAND |
|---|---|---|
| 0 | True | False |
| 1 | False | True |

In [100]:
```
cat_encoder.handle_unknown = "ignore"
cat_encoder.transform(df_test_unknown)
```

Out[100]:
```
array([[0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.]])
```

In [101]:
```
cat_encoder.feature_names_in_
```

Out[101]:
```
array(['ocean_proximity'], dtype=object)
```

In [102]:
```
cat_encoder.get_feature_names_out()
```

Out[102]:
```
array(['ocean_proximity_<1H OCEAN', 'ocean_proximity_INLAND',
       'ocean_proximity_ISLAND', 'ocean_proximity_NEAR BAY',
       'ocean_proximity_NEAR OCEAN'], dtype=object)
```

In [103]:
```
df_output = pd.DataFrame(cat_encoder.transform(df_test_unknown),
                         columns=cat_encoder.get_feature_names_out(),
                         index=df_test_unknown.index)
```

In [ ]:
```
df_output
```

| | ocean_proximity_<1H OCEAN | ocean_proximity_INLAND | ocean_proximity_ISL |
|---|---|---|---|
| **0** | 0.0 | 0.0 | 0.0 |
| **1** | 0.0 | 0.0 | 1.0 |

## Escalonamento de Características

Salvo raras exceções, em geral os algoritmos de ML não lidam bem com atributos com escalas muito diferentes. Na nossa base de dados, o número de cômodos varia entre 6 e 39.320, já a renda média varia entre 0 e 15. Sem escalonamento, a maioria dos algoritmos vai acabar sendo muito mais focado no número de cômodos, ignorando a renda média, que já sabemos ser um atributo muito importante.

Há duas formas de todos os atributos terem a mesma escala: *Escalonamento min-max* e *Padronização*.

In [104]:
```
#Aqui o MinMaxScaler vai transformar os atributos e deixar todos os
valores entre -1 e 1
from sklearn.preprocessing import MinMaxScaler

min_max_scaler = MinMaxScaler(feature_range=(-1, 1))
housing_num_min_max_scaled = min_max_scaler.fit_transform(housing_num)
```

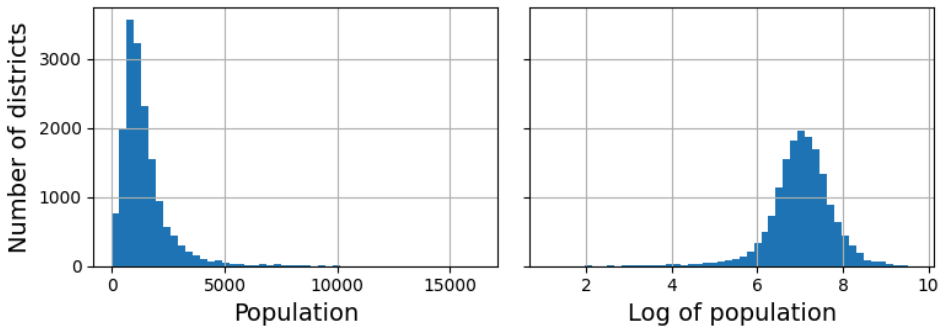
In [105]:
```
#Aqui o StardScaler vai fazer a Padronização dos atributos
from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
```

In [106]:
```
# Aqui temos um exemplo em que temos uma calda pesada, este código faz a
transformação dos dados, para ter uma distribuição mais simétrica

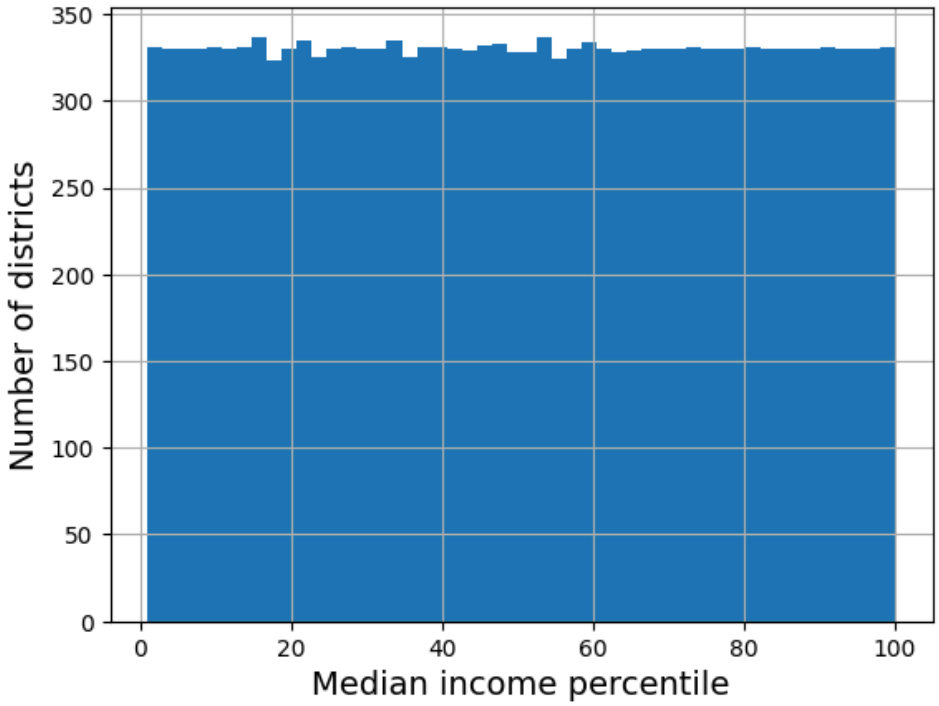
# Ao executar um escalonamento mix-max ou uma padronização nos dados, a
tendência é que os valores se aproximem mais do -1, prejudicando o
modelo
# Aqui, temos um exemplo em que primeiro  é feito uma melhoria na
distribuição dos dados

fig, axs = plt.subplots(1, 2, figsize=(8, 3), sharey=True)
housing["population"].hist(ax=axs[0], bins=50)
housing["population"].apply(np.log).hist(ax=axs[1], bins=50)
axs[0].set_xlabel("Population")
axs[1].set_xlabel("Log of population")
axs[0].set_ylabel("Number of districts")
save_fig("long_tail_plot")
plt.show()
```




E se substituirmos cada valor com seu percentual?

In [107]:
```
# Apenas mostrando como obter uma distribuição uniforme
percentiles = [np.percentile(housing["median_income"], p)
               for p in range(1, 100)]
flattened_median_income = pd.cut(housing["median_income"],
                                 bins=[-np.inf] + percentiles +
[np.inf],
                                 labels=range(1, 100 + 1))
flattened_median_income.hist(bins=50)
plt.xlabel("Median income percentile")
plt.ylabel("Number of districts")
plt.show()
# Observação: renda abaixo do 1o. percentual, são rotulados como 1, e
rendas acima do
# percentual 99, são rotulados como 100. Por isso a distribuição fica
entre 1 e 100
# (não de 0 a 100).
```




In [108]:
```
from sklearn.metrics.pairwise import rbf_kernel

age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]],
gamma=0.1)
```

In [109]:

```
# Exemplo em que é criado um novo atributo de similaridade.

# No caso, observando os dados de idade média dos imóveis, é observado
um pico em 35 anos
# Desta forma, pode se criar um atributo que mede a similaridade de um
imóvel qualquer com um típico imóvel de 35 anos

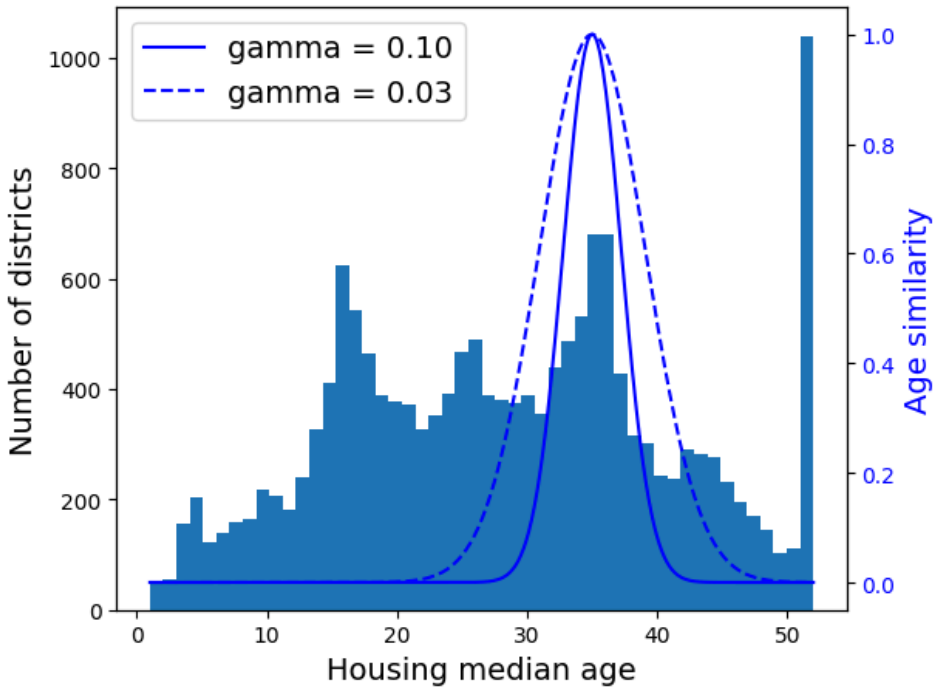
ages = np.linspace(housing["housing_median_age"].min(),
                   housing["housing_median_age"].max(),
                   500).reshape(-1, 1)
gamma1 = 0.1
gamma2 = 0.03
rbf1 = rbf_kernel(ages, [[35]], gamma=gamma1)
rbf2 = rbf_kernel(ages, [[35]], gamma=gamma2)

fig, ax1 = plt.subplots()

ax1.set_xlabel("Housing median age")
ax1.set_ylabel("Number of districts")
ax1.hist(housing["housing_median_age"], bins=50)

ax2 = ax1.twinx()  # create a twin axis that shares the same x-axis
color = "blue"
ax2.plot(ages, rbf1, color=color, label="gamma = 0.10")
ax2.plot(ages, rbf2, color=color, label="gamma = 0.03", linestyle="--")
ax2.tick_params(axis='y', labelcolor=color)
ax2.set_ylabel("Age similarity", color=color)

plt.legend(loc="upper left")
save_fig("age_similarity_plot")
plt.show()
```



# Modelo simples utilizando Regressão Linear e Inversão de Escala

**Fluxo Visual do Processo**

`
Preço real
↓ (StandardScaler)
Preço escalado
↓ (Treinamento)
Modelo aprende
↓ (Predição)
Preço escalado previsto
↓ (inverse_transform)
Preço real previsto
`

In [110]:
```
# Exemplo simplres de regressão linear, utilizando a Renda Média como
Entrada

from sklearn.linear_model import LinearRegression


# Escala os dados para o modelo
target_scaler = StandardScaler()
scaled_labels = target_scaler.fit_transform(housing_labels.to_frame())

model = LinearRegression()
model.fit(housing[["median_income"]], scaled_labels)
some_new_data = housing[["median_income"]].iloc[:5]  # utilizando como
se fossem dados novos

scaled_predictions = model.predict(some_new_data)

# Para evitar que a saida seja entre -1 e 1, o Scikit-Learn permite usar
a função inverse_transform, para retornar o valor na escala original
predictions = target_scaler.inverse_transform(scaled_predictions)
```

In [111]:
```
predictions
```

Out[111]:
```
array([[131997.15275877],
       [299359.35844434],
       [146023.37185694],
       [138840.33653057],
       [192016.61557639]])
```

In [112]:
```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(LinearRegression(),
                                   transformer=StandardScaler())
model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

In [113]:
```
predictions
```

Out[113]:
```
array([131997.15275877, 299359.35844434, 146023.37185694,
       138840.33653057,
              192016.61557639])
```

## Transformadores Personalizados

Criando Tranformadores Simples:

In [114]:
```
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
```

In [115]:
```
rbf_transformer = FunctionTransformer(rbf_kernel,
                                      kw_args=dict(Y=[[35.]],
gamma=0.1))
age_simil_35 =
rbf_transformer.transform(housing[["housing_median_age"]])
```

In [116]:
```
age_simil_35
```

Out[116]:
```
array([[2.81118530e-13],
       [8.20849986e-02],
       [6.70320046e-01],
       ...,
       [9.55316054e-22],
       [6.70320046e-01],
       [3.03539138e-04]])
```

In [117]:
```
sf_coords = 37.7749, -122.41
sf_transformer = FunctionTransformer(rbf_kernel,
                                     kw_args=dict(Y=[sf_coords],
gamma=0.1))
sf_simil = sf_transformer.transform(housing[["latitude", "longitude"]])
```

In [118]:
```
sf_simil
```

Out[118]:
```
array([[0.999927  ],
       [0.05258419],
       [0.94864161],
       ...,
       [0.00388525],
       [0.05038518],
       [0.99868067]])
```

In [119]:
```
ratio_transformer = FunctionTransformer(lambda X: X[:, [0]] / X[:, [1]])
ratio_transformer.transform(np.array([[1., 2.], [3., 4.]]))
```

Out[119]:
```
array([[0.5 ],
       [0.75]])
```

In [120]:
```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_array, check_is_fitted

class StandardScalerClone(BaseEstimator, TransformerMixin):
    def __init__(self, with_mean=True):  # no *args or **kwargs!
        self.with_mean = with_mean

    def fit(self, X, y=None):  # y is required even though we don't use
it
        X = check_array(X)  # checks that X is an array with finite
float values
        self.mean_ = X.mean(axis=0)
        self.scale_ = X.std(axis=0)
        self.n_features_in_ = X.shape[1]  # every estimator stores this
in fit()
        return self  # always return self!

    def transform(self, X):
        check_is_fitted(self)  # looks for learned attributes (with
trailing _)
        X = check_array(X)
        assert self.n_features_in_ == X.shape[1]
        if self.with_mean:
            X = X - self.mean_
        return X / self.scale_
```

In [121]:
```
from sklearn.cluster import KMeans

class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):
        self.n_clusters = n_clusters
        self.gamma = gamma
        self.random_state = random_state

    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters, n_init=10,
                              random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self  # always return self!

    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_,
gamma=self.gamma)

    def get_feature_names_out(self, names=None):
        return [f"Cluster {i} similarity" for i in
range(self.n_clusters)]
```

**Aviso:**

* Houve uma mudança no Scikit-Learn 1.3.0 que afetou o gerador de números aleatórios usado na inicialização do `KMeans`. Portanto, os resultados serão diferentes dos apresentados no livro se você utilizar o Scikit-Learn ≥ 1.3. Isso não é um problema, desde que você não espere que as saídas sejam perfeitamente idênticas.

* Ao longo deste notebook, quando o parâmetro `n_init` não foi definido ao criar um estimador `KMeans`, eu o defini explicitamente como `n_init=10` para evitar um aviso informando que o valor padrão desse hiperparâmetro mudará de 10 para `"auto"` no Scikit-Learn 1.4.

* Não ficou claro que a definição de `sample_weight=housing_labels` era apenas um exemplo e que esse parâmetro não é realmente utilizado durante o treinamento. Por isso, removi o argumento `sample_weight` abaixo, e a próxima figura corresponde aos clusters que de fato foram usados durante o treinamento.

In [122]:
```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1.,
random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude",
"longitude"]])
```

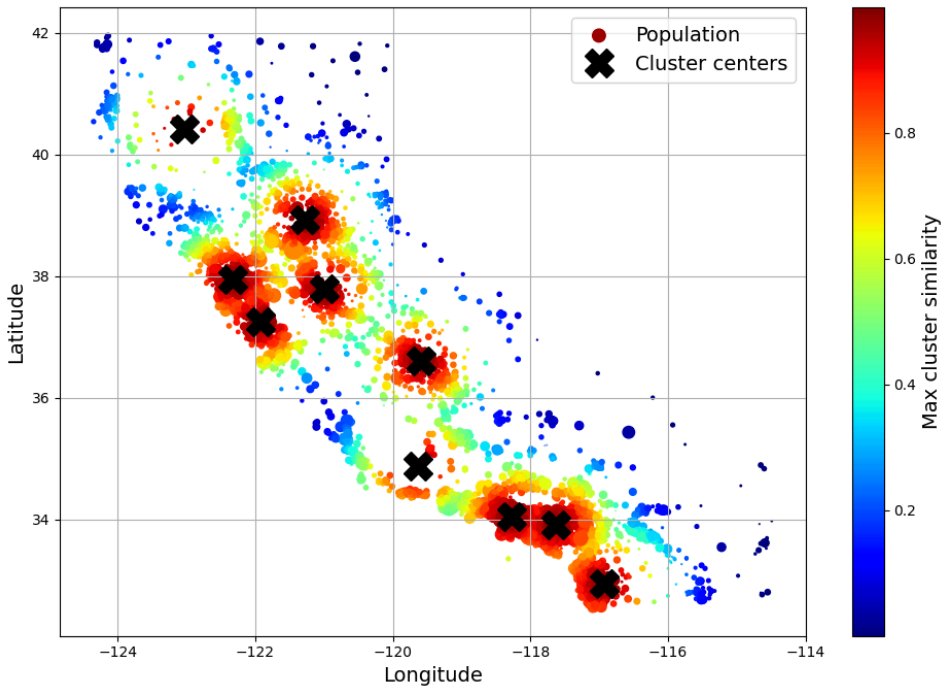
In [123]:
```
similarities[:3].round(2)
```

Out[123]:
```
array([[0.  , 0.97, 0.  , 0.  , 0.  , 0.08, 0.  , 0.  , 0.13,
        0.57],
       [0.12, 0.  , 0.98, 0.03, 0.  , 0.  , 0.  , 0.54, 0.  , 0.
        ],
       [0.  , 0.75, 0.  , 0.  , 0.  , 0.44, 0.  , 0.  , 0.27,
        0.28]])
```

In [124]:
```
# Exemplo utilizando Transformadores personalizados e Clusterização de
regiões

housing_renamed = housing.rename(columns={
    "latitude": "Latitude", "longitude": "Longitude",
    "population": "Population",
    "median_house_value": "Median house value (usd)"})
housing_renamed["Max cluster similarity"] = similarities.max(axis=1)

housing_renamed.plot(kind="scatter", x="Longitude", y="Latitude",
grid=True,
                     s=housing_renamed["Population"] / 100,
label="Population",
                     c="Max cluster similarity",
                     cmap="jet", colorbar=True,
                     legend=True, sharex=False, figsize=(10, 7))
plt.plot(cluster_simil.kmeans_.cluster_centers_[:, 1],
         cluster_simil.kmeans_.cluster_centers_[:, 0],
         linestyle="", color="black", marker="X", markersize=20,
         label="Cluster centers")
plt.legend(loc="upper right")
save_fig("district_cluster_plot")
plt.show()
```



# Pipelines de Transformação

Durante o processo de limpeza, preparação e transformação dos dados, inúmeras etapas precisam ser executadas em uma ordem correta. A Scikit-Learn prove a classe `Pipeline` para ajudar a organizar a sequência de transformações.

Agora, vamos construir um pequeno pipeline para pré-processar os atributos numéricos:

In [125]:
```
#Pipeline criado informando os nomes das Etapas manualmente
from sklearn.pipeline import Pipeline

num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("standardize", StandardScaler()),
])
```

In [126]:
```
#Pipeline criado utilizando o "make_pipeline" que nomeara as etapas por convenção do Scikit-Learn
from sklearn.pipeline import make_pipeline

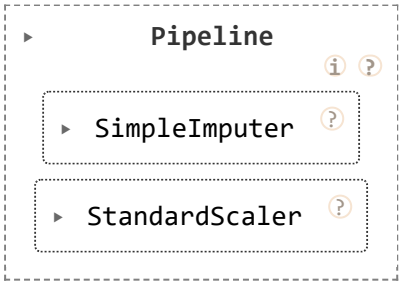
num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
```

In [127]:
```
from sklearn import set_config

set_config(display='diagram')

num_pipeline
```

Out[127]:




In [128]:
```
housing_num_prepared = num_pipeline.fit_transform(housing_num)
housing_num_prepared[:2].round(2)
```

Out[128]:
```
array([[-1.42,  1.01,  1.86,  0.31,  1.37,  0.14,  1.39, -0.94],
       [ 0.6 , -0.7 ,  0.91, -0.31, -0.44, -0.69, -0.37,  1.17]])
```

In [129]:
```
def monkey_patch_get_signature_names_out():
    """Monkey patch - Ajusta algumas classes que não implementam
get_feature_names_out()
       corretamente no Scikit-Learn 1.0.*."""
    from inspect import Signature, signature, Parameter
    import pandas as pd
    from sklearn.impute import SimpleImputer
    from sklearn.pipeline import make_pipeline, Pipeline
    from sklearn.preprocessing import FunctionTransformer,
StandardScaler

    default_get_feature_names_out = StandardScaler.get_feature_names_out

    if not hasattr(SimpleImputer, "get_feature_names_out"):
      print("Monkey-patching SimpleImputer.get_feature_names_out()")
      SimpleImputer.get_feature_names_out =
default_get_feature_names_out

    if not hasattr(FunctionTransformer, "get_feature_names_out"):
        print("Monkey-patching
FunctionTransformer.get_feature_names_out()")
        orig_init = FunctionTransformer.__init__
        orig_sig = signature(orig_init)

        def __init__(*args, feature_names_out=None, **kwargs):
            orig_sig.bind(*args, **kwargs)
            orig_init(*args, **kwargs)
            args[0].feature_names_out = feature_names_out

        __init__.__signature__ = Signature(
            list(signature(orig_init).parameters.values()) + [
                Parameter("feature_names_out", Parameter.KEYWORD_ONLY)])

        def get_feature_names_out(self, names=None):
            if callable(self.feature_names_out):
                return self.feature_names_out(self, names)
            assert self.feature_names_out == "one-to-one"
            return default_get_feature_names_out(self, names)

        FunctionTransformer.__init__ = __init__
        FunctionTransformer.get_feature_names_out =
get_feature_names_out

monkey_patch_get_signature_names_out()
```

In [130]:
```
df_housing_num_prepared = pd.DataFrame(
    housing_num_prepared, columns=num_pipeline.get_feature_names_out(),
    index=housing_num.index)
```

In [131]:
```
# Exibindo alguns dados após a execução atual do Pipeline e ajuste dos nomes das propriedades
df_housing_num_prepared.head(5)
```

Out[131]:

| | longitude | latitude | housing_median_age | total_rooms | tot |
|---|---|---|---|---|---|
| **13096** | -1.423037 | 1.013606 | 1.861119 | 0.311912 | 1.3 |
| **14973** | 0.596394 | -0.702103 | 0.907630 | -0.308620 | -0.4 |
| **3785** | -1.203098 | 1.276119 | 0.351428 | -0.712240 | -0.7 |
| **14689** | 1.231216 | -0.884924 | -0.919891 | 0.702262 | 0.7 |
| **20507** | 0.711362 | -0.875549 | 0.589800 | 0.790125 | 1.5 |

In [132]:
```
num_pipeline.steps
```

Out[132]:
```
[('simpleimputer', SimpleImputer(strategy='median')),
 ('standardscaler', StandardScaler())]
```

In [133]:
```
num_pipeline[1]
```

Out[133]:
```
StandardScaler
StandardScaler()
```

In [134]:
```
num_pipeline[:-1]
```

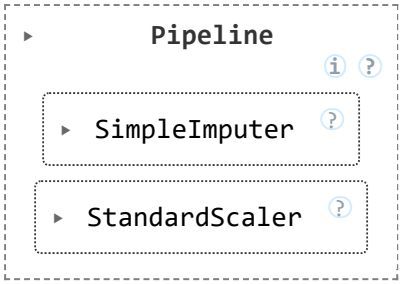
Out[134]:
```
Pipeline
  SimpleImputer
```

In [135]:
```
num_pipeline.named_steps["simpleimputer"]
```

Out[135]:
```
SimpleImputer
SimpleImputer(strategy='median')
```

In [136]:
```
num_pipeline.set_params(simpleimputer__strategy="median")
```

Out[136]:




In [137]:
```
# Até o momento, foram aplicados transformadores somente nos campos
numéricos

# Agora, utilizaremos um ColumnTransformer, para aplicar as
transformações a
# todas as colunas de uma vez
from sklearn.compose import ColumnTransformer

num_attribs = ["longitude", "latitude", "housing_median_age",
"total_rooms",
               "total_bedrooms", "population", "households",
"median_income"]
cat_attribs = ["ocean_proximity"]

cat_pipeline = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))

preprocessing = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", cat_pipeline, cat_attribs),
])
```

In [138]:
```
from sklearn.compose import make_column_selector,
make_column_transformer

preprocessing = make_column_transformer(
    (num_pipeline, make_column_selector(dtype_include=np.number)),
    (cat_pipeline, make_column_selector(dtype_include=object)),
)
```

In [139]:
```
housing_prepared = preprocessing.fit_transform(housing)
```

In [140]:
```
# extra code – Mostrando que podemos ter uma saída de DataFrame, se
quisermos
housing_prepared_fr = pd.DataFrame(
    housing_prepared,
    columns=preprocessing.get_feature_names_out(),
    index=housing.index)
housing_prepared_fr.head(5)
```

Out[140]:

| | pipeline-1__longitude | pipeline-1__latitude | pipeline-1__housing_median_age | pipelin 1__total_roor |
|---|---|---|---|---|
| **13096** | -1.423037 | 1.013606 | 1.861119 | 0.311912 |
| **14973** | 0.596394 | -0.702103 | 0.907630 | -0.308620 |
| **3785** | -1.203098 | 1.276119 | 0.351428 | -0.712240 |
| **14689** | 1.231216 | -0.884924 | -0.919891 | 0.702262 |
| **20507** | 0.711362 | -0.875549 | 0.589800 | 0.790125 |

# Pipeline Consolidado

Aqui temos um resumo de todas as transformações que o Pipeline abaixo executará nos dados:
- Os Valores Ausentes nos atributos numéricos, serão substituidos pela média
- Os Valores Ausentes nos atributos categóricos, serão substituidos pela categoria mais frequente
- O atributo categórico `ocean_proximity` será codificado com one-hot
- Serão calculadas as seguintes categorias de proporção: `beedrooms_ratio`, `rooms_per_house` e `people_per_house`.
- Serão criadas características de similaridade de cluster geografico a partir de `latitude` e `longitude`
- As características com uma calda longa, serão substituidas pelo seu logaritmo, para ter uma distribuição mais uniforme
- Todas as características numéricas serão padronizadas, para que todas as características tenham aproximadamente a mesma escala

In [141]:
```
def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

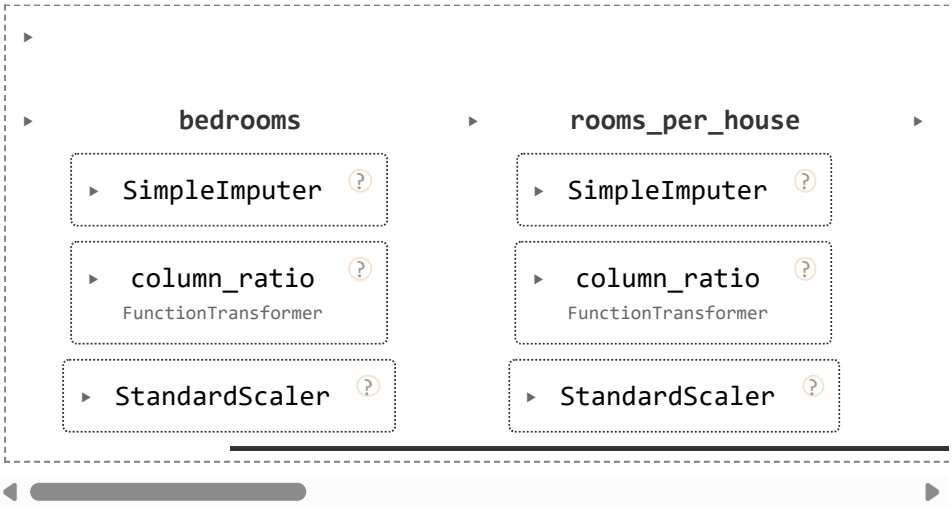
def ratio_name(function_transformer, feature_names_in):
    return ["ratio"]  # feature names out

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler())

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1.,
random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                                     StandardScaler())
preprocessing = ColumnTransformer([
        ("bedrooms", ratio_pipeline(), ["total_bedrooms",
"total_rooms"]),
        ("rooms_per_house", ratio_pipeline(), ["total_rooms",
"households"]),
        ("people_per_house", ratio_pipeline(), ["population",
"households"]),
        ("log", log_pipeline, ["total_bedrooms", "total_rooms",
"population",
                               "households", "median_income"]),
        ("geo", cluster_simil, ["latitude", "longitude"]),
        ("cat", cat_pipeline,
make_column_selector(dtype_include=object)),
    ],
    remainder=default_num_pipeline)  # one column remaining:
housing_median_age
```

In [142]:
```
preprocessing
```

Out[142]:



In [143]:
```
housing_prepared = preprocessing.fit_transform(housing)
housing_prepared.shape
```

Out[143]: (16512, 24)

In [144]:
```
preprocessing.get_feature_names_out()
```

Out[144]:
```
array(['bedrooms__ratio', 'rooms_per_house__ratio',
       'people_per_house__ratio', 'log__total_bedrooms',
       'log__total_rooms', 'log__population', 'log__households',
       'log__median_income', 'geo__Cluster 0 similarity',
       'geo__Cluster 1 similarity', 'geo__Cluster 2 similarity',
       'geo__Cluster 3 similarity', 'geo__Cluster 4 similarity',
       'geo__Cluster 5 similarity', 'geo__Cluster 6 similarity',
       'geo__Cluster 7 similarity', 'geo__Cluster 8 similarity',
       'geo__Cluster 9 similarity', 'cat__ocean_proximity_<1H
OCEAN',
       'cat__ocean_proximity_INLAND',
'cat__ocean_proximity_ISLAND',
       'cat__ocean_proximity_NEAR BAY', 'cat__ocean_proximity_NEAR
OCEAN',
       'remainder__housing_median_age'], dtype=object)
```

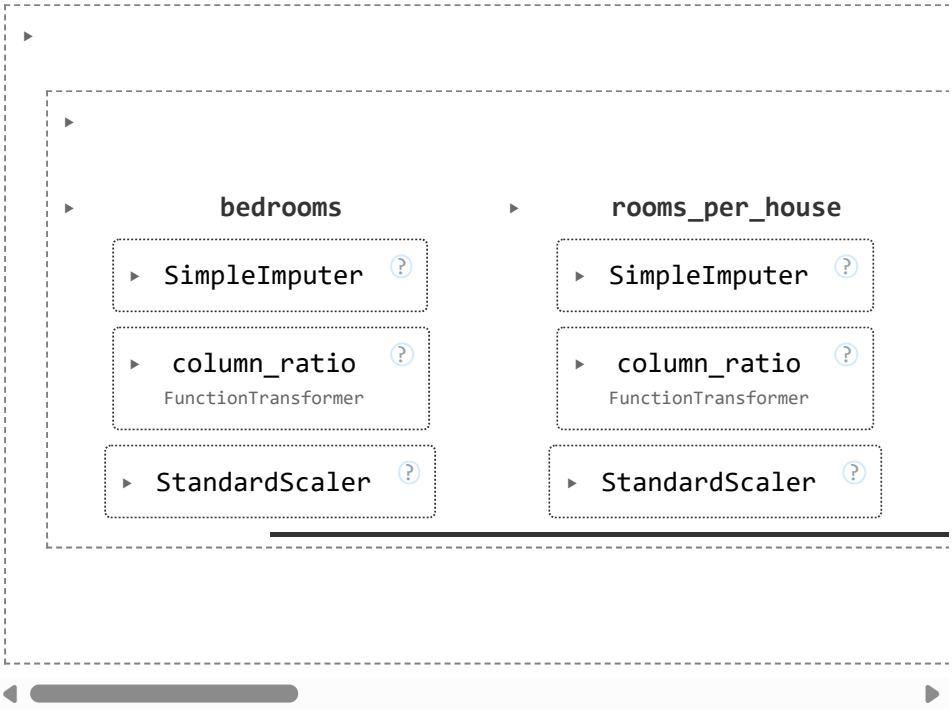
# Selecionando e Treinando um Modelo

## Treinando e avaliando o Conjunto de Treinamento (Training Set)

In [145]:
```
from sklearn.linear_model import LinearRegression

lin_reg = make_pipeline(preprocessing, LinearRegression())
lin_reg.fit(housing, housing_labels)
```

Out[145]:




Vamos experimentar o pipeline de preprocessamento em algumas instancias de treino:

In [146]:
```
housing_predictions = lin_reg.predict(housing)
housing_predictions[:5].round(-2)  # -2 = rounded to the nearest hundred
```

Out[146]:
```
array([242800., 375900., 127500.,  99400., 324600.])
```

E comparar contra os valores atuais:

In [147]:
```
housing_labels.iloc[:5].values
```

Out[147]:
```
array([458300., 483800., 101700.,  96100., 361800.])
```

In [148]:
```
# Calculando a proporção de erro dos valores
error_ratios = housing_predictions[:5].round(-2) /
housing_labels.iloc[:5].values - 1
print(", ".join([f"{100 * ratio:.1f}%" for ratio in error_ratios]))
```

```
-47.0%, -22.3%, 25.4%, 3.4%, -10.3%
```

## Calculando o RMSE do modelo

**Aviso**: Em versões mais recentes do Scikit-Learn, você deve usar `root_mean_squared_error(labels, predictions)` para calcular o RMSE, em vez de `mean_squared_error(labels, predictions, squared=False)`.
O bloco `try` / `except` a seguir tenta importar `root_mean_squared_error` e, caso isso falhe, simplesmente define essa função.

In [149]:
```
try:
    from sklearn.metrics import root_mean_squared_error
except ImportError:
    from sklearn.metrics import mean_squared_error

    def root_mean_squared_error(labels, predictions):
        return mean_squared_error(labels, predictions, squared=False)

lin_rmse = root_mean_squared_error(housing_labels, housing_predictions)
lin_rmse
```

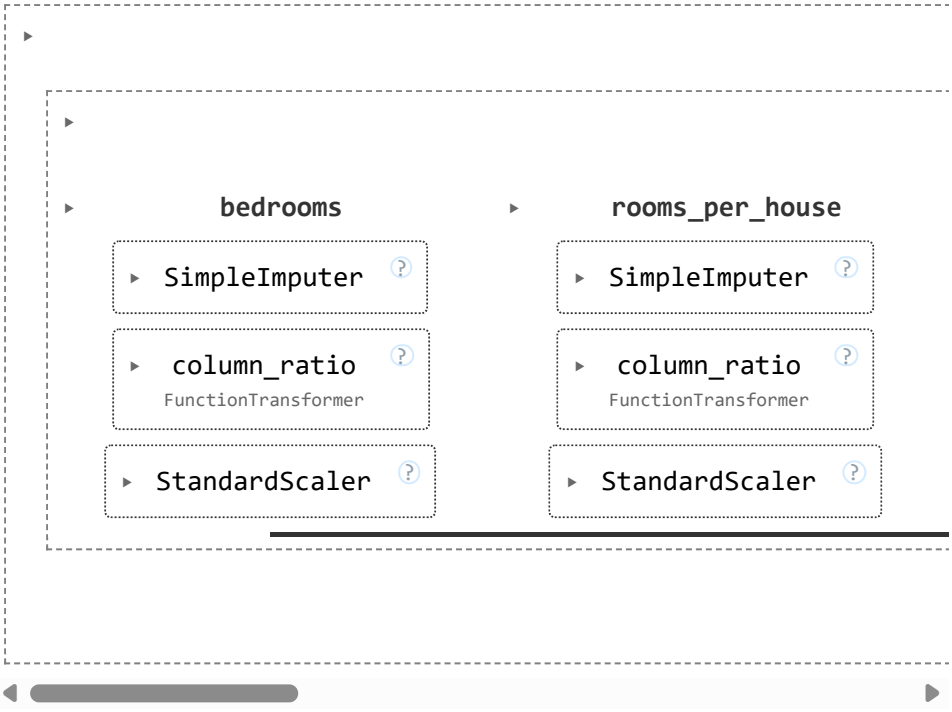
Out[149]: 68647.95686706658

### Aplicando o modelo de Regressor por Árvore de Decisão

In [150]:
```
from sklearn.tree import DecisionTreeRegressor

tree_reg = make_pipeline(preprocessing,
DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)
```

Out[150]:




In [151]:
```
housing_predictions = tree_reg.predict(housing)
tree_rmse = root_mean_squared_error(housing_labels, housing_predictions)
tree_rmse
```

Out[151]: 0.0

RMSE calculado com "0", então não houve nenhum erro? Provavelmente o modelo foi sobreajustado mal aos dados.

## Melhor avaliação com Validação Cruzada

Aqui o Conjunto de Testes será dividido em 10 subconjuntos, a partir desta divisão, serão treinados 10 modelos, utilizando 1 conjunto para validação e os outros 9 para treinamento.

Ao final, vai medir todos os modelos com RMSE.

In [152]:
```
from sklearn.model_selection import cross_val_score

tree_rmses = -cross_val_score(tree_reg, housing, housing_labels,
                              scoring="neg_root_mean_squared_error",
cv=10)
```

In [153]:
```
pd.Series(tree_rmses).describe()
```

Out[153]:

| | 0 |
|---|---|
| **count** | 10.000000 |
| **mean** | 66366.983603 |
| **std** | 1976.844743 |
| **min** | 63557.655007 |
| **25%** | 65004.623899 |
| **50%** | 65886.897085 |
| **75%** | 68129.026040 |
| **max** | 69530.301101 |

**dtype:** float64

In [154]:
```
# Fazendo a Validação Cruzada, calculando RMSE para o modelo de
Regressão Linear
lin_rmses = -cross_val_score(lin_reg, housing, housing_labels,
                             scoring="neg_root_mean_squared_error",
cv=10)
pd.Series(lin_rmses).describe()
```

Out[154]:

| | 0 |
|---|---|
| **count** | 10.000000 |
| **mean** | 69847.923224 |
| **std** | 4078.407329 |
| **min** | 65659.761079 |
| **25%** | 68088.799156 |
| **50%** | 68697.591463 |
| **75%** | 69800.966364 |
| **max** | 80685.254832 |

**dtype:** float64

### Aplicando o modelo de Regressão por Floresta Aleatória

**Aviso:** O código abaixo pode levar alguns minutos para executar:

In [155]:
```
from sklearn.ensemble import RandomForestRegressor

forest_reg = make_pipeline(preprocessing,
                           RandomForestRegressor(random_state=42))
forest_rmses = -cross_val_score(forest_reg, housing, housing_labels,
                                scoring="neg_root_mean_squared_error",
cv=10)
```

In [156]:
```
pd.Series(forest_rmses).describe()
```

Out[156]:

| | 0 |
|---|---|
| **count** | 10.000000 |
| **mean** | 46938.209246 |
| **std** | 1018.397196 |
| **min** | 45522.649195 |
| **25%** | 46291.334639 |
| **50%** | 47021.703303 |
| **75%** | 47321.521991 |
| **max** | 49140.832210 |

**dtype:** float64

Vamos comparar o RMSE medido usando validação cruzada (o “erro de validação”) com o RMSE medido no conjunto de treinamento (o “erro de treinamento”):

In [157]:
```
forest_reg.fit(housing, housing_labels)
housing_predictions = forest_reg.predict(housing)
forest_rmse = root_mean_squared_error(housing_labels,
housing_predictions)
forest_rmse
```

Out[157]: 17521.565358779884

O erro de treinamento é muito menor do que o erro de validação, o que geralmente significa que o modelo se ajustou demais (overfitting) ao conjunto de treinamento.

Outra possível explicação poderia ser que exista uma discrepância entre os dados de treinamento e os dados de validação, mas não é o caso aqui, já que ambos vieram do mesmo conjunto de dados, que foi embaralhado e dividido em duas partes.

# Aperfeiçoe o seu Modelo

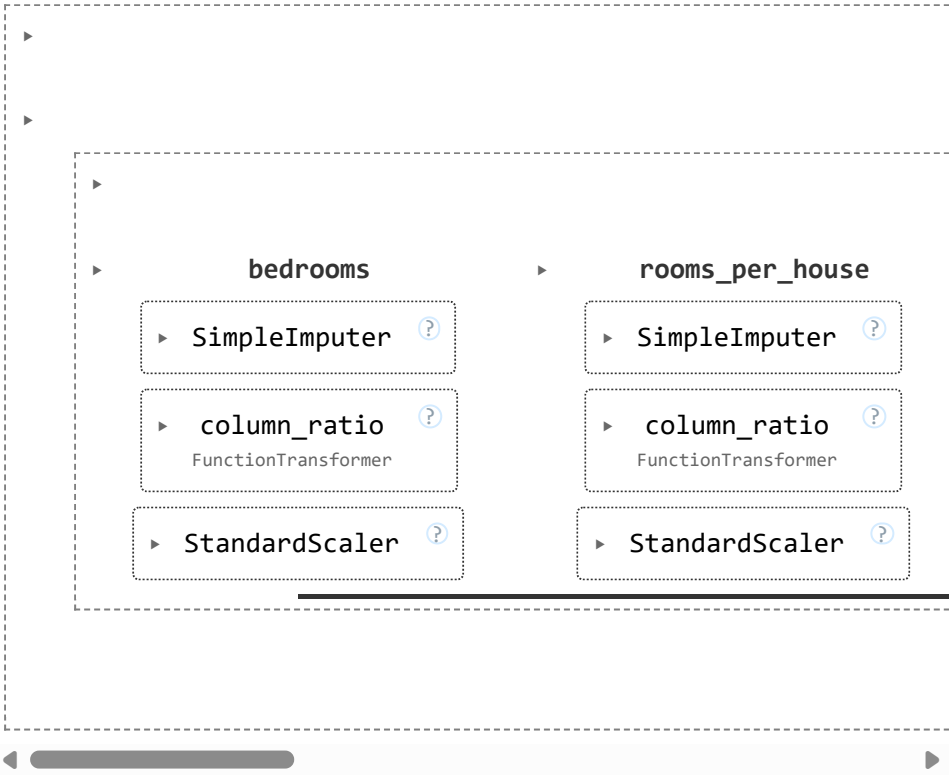
## Grid Search

**Aviso:** O código abaixo pode levar alguns minutos para executar:

In [158]:
```
from sklearn.model_selection import GridSearchCV

full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("random_forest", RandomForestRegressor(random_state=42)),
])
param_grid = [
    {'preprocessing__geo__n_clusters': [5, 8, 10],
     'random_forest__max_features': [4, 6, 8]},
    {'preprocessing__geo__n_clusters': [10, 15],
     'random_forest__max_features': [6, 8, 10]},
]
grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
                           scoring='neg_root_mean_squared_error')
grid_search.fit(housing, housing_labels)
```

Out[158]:




É possível acessar a lista completa de hiperparametros para ajuste através do comando `full_pipeline.get_params().keys()` :

In [159]:
```
# extra code – shows part of the output of get_params().keys()
print(str(full_pipeline.get_params().keys())[:1000] + "...")
```

```
dict_keys(['memory', 'steps', 'transform_input', 'verbose',
'preprocessing', 'random_forest',
'preprocessing__force_int_remainder_cols',
'preprocessing__n_jobs', 'preprocessing__remainder__memory',
'preprocessing__remainder__steps',
'preprocessing__remainder__transform_input',
'preprocessing__remainder__verbose',
'preprocessing__remainder__simpleimputer',
'preprocessing__remainder__standardscaler',
'preprocessing__remainder__simpleimputer__add_indicator',
'preprocessing__remainder__simpleimputer__copy',
'preprocessing__remainder__simpleimputer__fill_value',
'preprocessing__remainder__simpleimputer__keep_empty_features',
'preprocessing__remainder__simpleimputer__missing_values',
'preprocessing__remainder__simpleimputer__strategy',
'preprocessing__remainder__standardscaler__copy',
'preprocessing__remainder__standardscaler__with_mean',
'preprocessing__remainder__standardscaler__with_std',
'preprocessing__remainder', 'preprocessing__sparse_threshold',
'preprocessing__transformer_weights', ...
```

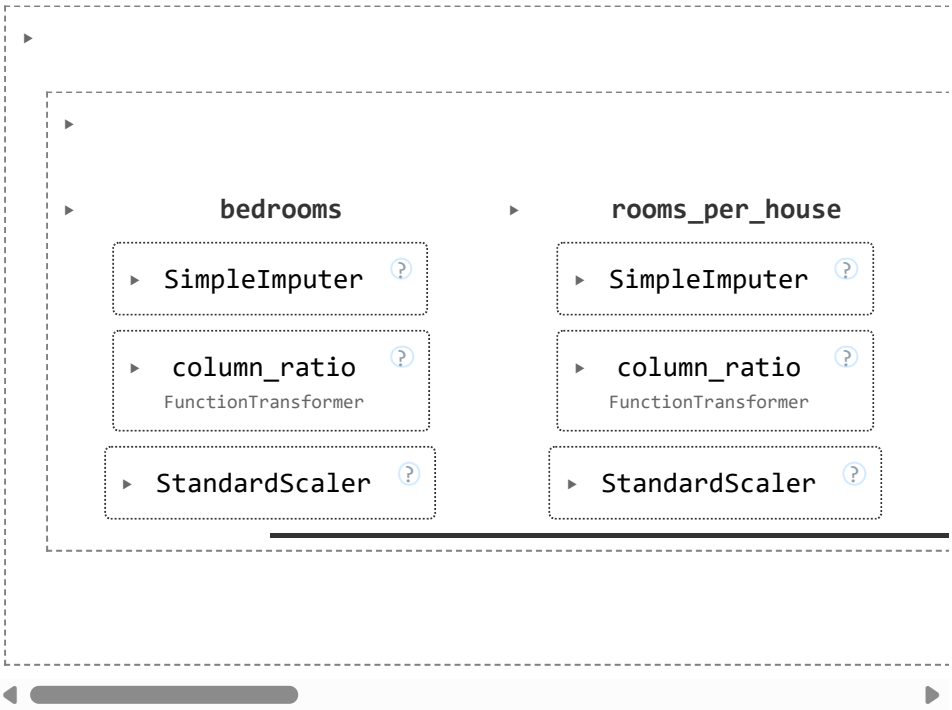
A melhor combinação de hiperparametros encontrada:

In [160]:
```
grid_search.best_params_
```

Out[160]:
```
{'preprocessing__geo__n_clusters': 15,
'random_forest__max_features': 6}
```

In [161]:
```
grid_search.best_estimator_
```

Out[161]:



Aqui podemos ver o melhor score de cada combinação de hiperparâmetros testados durante a execução do Grid Search:

In [162]:
```
cv_res = pd.DataFrame(grid_search.cv_results_)
cv_res.sort_values(by="mean_test_score", ascending=False, inplace=True)

# extra code – these few lines of code just make the DataFrame look
nicer
cv_res = cv_res[["param_preprocessing__geo__n_clusters",
                 "param_random_forest__max_features",
"split0_test_score",
                 "split1_test_score", "split2_test_score",
"mean_test_score"]]
score_cols = ["split0", "split1", "split2", "mean_test_rmse"]
cv_res.columns = ["n_clusters", "max_features"] + score_cols
cv_res[score_cols] = -cv_res[score_cols].round().astype(np.int64)

cv_res.head()
```

Out[162]:

| | n_clusters | max_features | split0 | split1 | split2 | mean_test_rms |
|---|---|---|---|---|---|---|
| **12** | 15 | 6 | 43007 | 43683 | 44158 | 43616 |
| **13** | 15 | 8 | 43697 | 44011 | 44819 | 44176 |
| **7** | 10 | 6 | 43710 | 44133 | 45227 | 44356 |
| **9** | 10 | 6 | 43710 | 44133 | 45227 | 44356 |
| **6** | 10 | 4 | 43798 | 44233 | 45100 | 44377 |

## Randomized Search

In [163]:
```
from sklearn.experimental import enable_halving_search_cv
from sklearn.model_selection import HalvingRandomSearchCV
```

Tente 30 ( `n_iter × cv` ) combinações aleatórias de hiperparâmetros:

**Aviso:** O código abaixo pode levar alguns minutos para executar:

In [164]:
```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distribs = {'preprocessing__geo__n_clusters': randint(low=3,
high=50),
                  'random_forest__max_features': randint(low=2,
high=20)}

rnd_search = RandomizedSearchCV(
    full_pipeline, param_distributions=param_distribs, n_iter=10, cv=3,
    scoring='neg_root_mean_squared_error', random_state=42)

rnd_search.fit(housing, housing_labels)
```

Out[164]:



In [165]:
```
# Exibe os resultados do random search
cv_res = pd.DataFrame(rnd_search.cv_results_)
cv_res.sort_values(by="mean_test_score", ascending=False, inplace=True)
cv_res = cv_res[["param_preprocessing__geo__n_clusters",
                 "param_random_forest__max_features",
"split0_test_score",
                 "split1_test_score", "split2_test_score",
"mean_test_score"]]
cv_res.columns = ["n_clusters", "max_features"] + score_cols
cv_res[score_cols] = -cv_res[score_cols].round().astype(np.int64)
cv_res.head()
```

Out[165]:

| | n_clusters | max_features | split0 | split1 | split2 | mean_test_rmse |
|---|---|---|---|---|---|---|
| **1** | 45 | 9 | 41224 | 42385 | 42712 | 42107 |
| **8** | 32 | 7 | 41374 | 42579 | 43230 | 42394 |
| **0** | 41 | 16 | 42397 | 42640 | 43175 | 42738 |
| **5** | 42 | 4 | 41874 | 43348 | 43690 | 42971 |
| **2** | 23 | 8 | 42421 | 43138 | 43585 | 43048 |

**Seção bônus: como escolher a distribuição de amostragem para um hiperparâmetro**

- `scipy.stats.randint(a, b+1)` : para hiperparâmetros com valores discretos que variam de a até b, e em que todos os valores nesse intervalo parecem igualmente prováveis.
- `scipy.stats.uniform(a, b)` : muito semelhante ao anterior, mas para hiperparâmetros contínuos.
- `scipy.stats.geom(1 / scale)` : para valores discretos, quando você deseja amostrar aproximadamente em uma determinada escala.
Por exemplo, com scale=1000, a maioria das amostras ficará nessa faixa, mas cerca de 10% das amostras serão menores que 100 e cerca de 10% serão maiores que 2300.
- `scipy.stats.expon(scale)` : equivalente contínuo do geom. Basta definir scale como o valor mais provável.
- `scipy.stats.loguniform(a, b)` : quando você tem pouquíssima ideia de qual é a escala ideal do hiperparâmetro.
Se você definir a=0.01 e b=100, então será tão provável sortear um valor entre 0.01 e 0.1 quanto um valor entre 10 e 100.

Aqui estão os gráficos das funções de massa de probabilidade (para variáveis discretas) e das funções de densidade de probabilidade (para variáveis contínuas) para `randint()`, `uniform()`, `geom()` e `expon()`:

In [166]:

```
# Mostrando algumas combinações que você pode usar em um Randomized
Search

from scipy.stats import randint, uniform, geom, expon

xs1 = np.arange(0, 7 + 1)
randint_distrib = randint(0, 7 + 1).pmf(xs1)

xs2 = np.linspace(0, 7, 500)
uniform_distrib = uniform(0, 7).pdf(xs2)

xs3 = np.arange(0, 7 + 1)
geom_distrib = geom(0.5).pmf(xs3)

xs4 = np.linspace(0, 7, 500)
expon_distrib = expon(scale=1).pdf(xs4)

plt.figure(figsize=(12, 7))

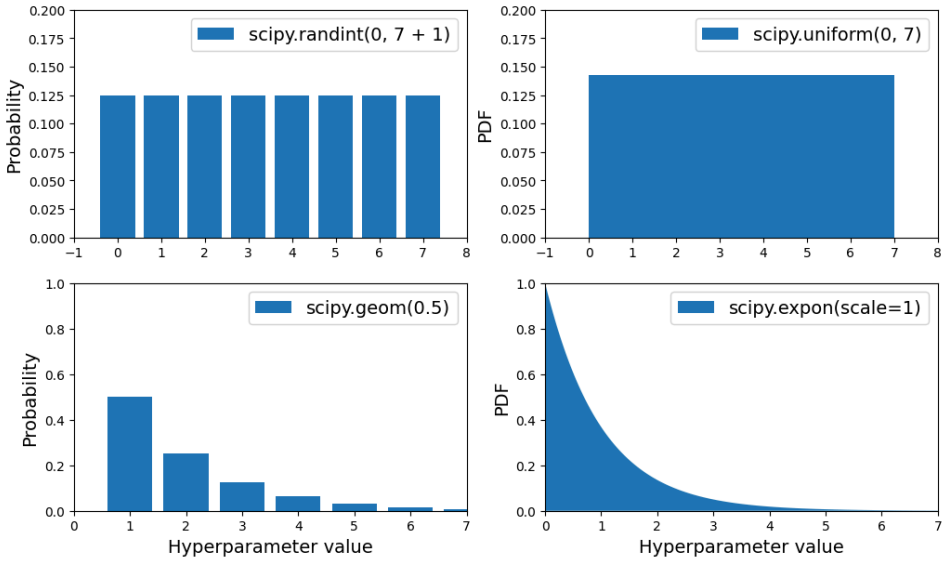
plt.subplot(2, 2, 1)
plt.bar(xs1, randint_distrib, label="scipy.randint(0, 7 + 1)")
plt.ylabel("Probability")
plt.legend()
plt.axis([-1, 8, 0, 0.2])

plt.subplot(2, 2, 2)
plt.fill_between(xs2, uniform_distrib, label="scipy.uniform(0, 7)")
plt.ylabel("PDF")
plt.legend()
plt.axis([-1, 8, 0, 0.2])

plt.subplot(2, 2, 3)
plt.bar(xs3, geom_distrib, label="scipy.geom(0.5)")
plt.xlabel("Hyperparameter value")
plt.ylabel("Probability")
plt.legend()
plt.axis([0, 7, 0, 1])

plt.subplot(2, 2, 4)
plt.fill_between(xs4, expon_distrib, label="scipy.expon(scale=1)")
plt.xlabel("Hyperparameter value")
plt.ylabel("PDF")
plt.legend()
plt.axis([0, 7, 0, 1])

plt.show()
```




Aqui estão as funções densidade de probabilidade (PDF) para `expon()` e `loguniform()` (coluna da esquerda), bem como a PDF de log(X) (coluna da direita). A coluna da direita mostra a distribuição das escalas dos hiperparâmetros.

É possível ver que `expon()` favorece hiperparâmetros com aproximadamente a escala desejada, com uma cauda mais longa em direção às escalas menores. Já `loguniform()` não favorece nenhuma escala específica: todas as escalas são igualmente prováveis.

In [167]:

```
# Mostrando a diferença entre expon e loguniform

from scipy.stats import loguniform

xs1 = np.linspace(0, 7, 500)
expon_distrib = expon(scale=1).pdf(xs1)

log_xs2 = np.linspace(-5, 3, 500)
log_expon_distrib = np.exp(log_xs2 - np.exp(log_xs2))

xs3 = np.linspace(0.001, 1000, 500)
loguniform_distrib = loguniform(0.001, 1000).pdf(xs3)

log_xs4 = np.linspace(np.log(0.001), np.log(1000), 500)
log_loguniform_distrib = uniform(np.log(0.001),
np.log(1000)).pdf(log_xs4)

plt.figure(figsize=(12, 7))

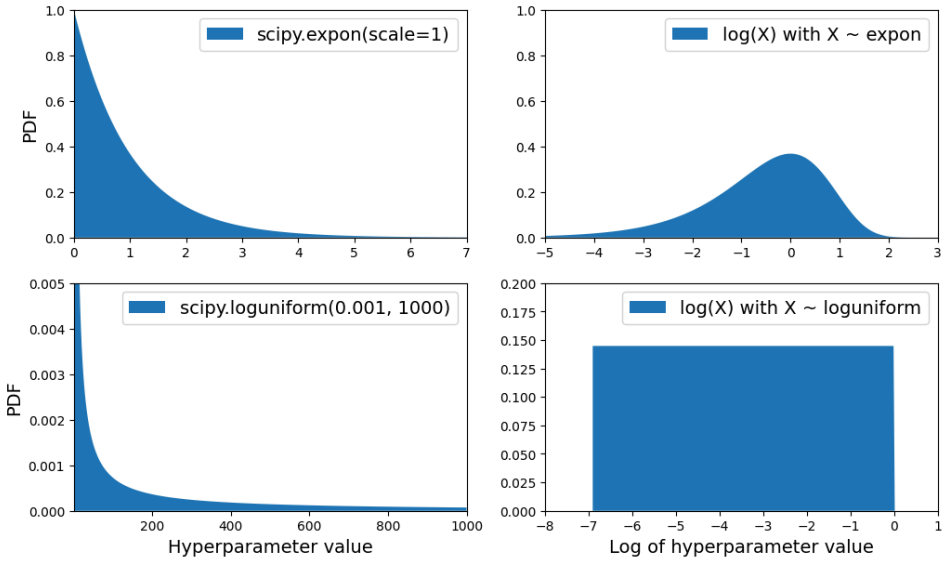
plt.subplot(2, 2, 1)
plt.fill_between(xs1, expon_distrib,
                 label="scipy.expon(scale=1)")
plt.ylabel("PDF")
plt.legend()
plt.axis([0, 7, 0, 1])

plt.subplot(2, 2, 2)
plt.fill_between(log_xs2, log_expon_distrib,
                 label="log(X) with X ~ expon")
plt.legend()
plt.axis([-5, 3, 0, 1])

plt.subplot(2, 2, 3)
plt.fill_between(xs3, loguniform_distrib,
                 label="scipy.loguniform(0.001, 1000)")
plt.xlabel("Hyperparameter value")
plt.ylabel("PDF")
plt.legend()
plt.axis([0.001, 1000, 0, 0.005])

plt.subplot(2, 2, 4)
plt.fill_between(log_xs4, log_loguniform_distrib,
                 label="log(X) with X ~ loguniform")
plt.xlabel("Log of hyperparameter value")
plt.legend()
plt.axis([-8, 1, 0, 0.2])

plt.show()
```




## Analisando os Melhores Modelos e seus Erros

In [168]:
```
final_model = rnd_search.best_estimator_  # inclui pré-processamento
feature_importances = final_model["random_forest"].feature_importances_
feature_importances.round(2)
```

Out[168]:
```
array([0.06, 0.05, 0.05, 0.01, 0.01, 0.01, 0.01, 0.19, 0.01, 0.01,
0.02,
       0.04, 0.01, 0.  , 0.02, 0.01, 0.01, 0.  , 0.02, 0.01, 0.  ,
0.01,
       0.01, 0.01, 0.01, 0.02, 0.02, 0.01, 0.01, 0.01, 0.02, 0.01,
0.01,
       0.01, 0.01, 0.01, 0.01, 0.  , 0.01, 0.01, 0.01, 0.01, 0.01,
0.02,
       0.01, 0.01, 0.01, 0.01, 0.02, 0.02, 0.01, 0.01, 0.01, 0.  ,
0.08,
       0.  , 0.  , 0.  , 0.01])
```

In [169]:
```
sorted(zip(feature_importances,
           final_model["preprocessing"].get_feature_names_out()),
           reverse=True)
```

Out[169]:
```
[(np.float64(0.18883049114642353), 'log__median_income'),
 (np.float64(0.07550690165952419), 'cat__ocean_proximity_INLAND'),
 (np.float64(0.06427087748676082), 'bedrooms__ratio'),
 (np.float64(0.0521695509969851), 'rooms_per_house__ratio'),
 (np.float64(0.04664258696240615), 'people_per_house__ratio'),
 (np.float64(0.04244311259449021), 'geo__Cluster 3 similarity'),
 (np.float64(0.02329106317954673), 'geo__Cluster 17 similarity'),
 (np.float64(0.022649484211911694), 'geo__Cluster 18 similarity'),
 (np.float64(0.02197243576680579), 'geo__Cluster 22 similarity'),
 (np.float64(0.01881537204558837), 'geo__Cluster 40 similarity'),
 (np.float64(0.01766998676736003), 'geo__Cluster 10 similarity'),
 (np.float64(0.01754694953848625), 'geo__Cluster 35 similarity'),
 (np.float64(0.017274903717750808), 'geo__Cluster 41 similarity'),
 (np.float64(0.016055663107420095), 'geo__Cluster 6 similarity'),
 (np.float64(0.015811758952117744), 'geo__Cluster 2 similarity'),
 (np.float64(0.01477755940463033), 'geo__Cluster 32 similarity'),
 (np.float64(0.01384559635134935), 'geo__Cluster 43 similarity'),
 (np.float64(0.013738038212939275), 'geo__Cluster 21 similarity'),
 (np.float64(0.013012561860104492), 'geo__Cluster 39 similarity'),
 (np.float64(0.012140211177608811), 'geo__Cluster 26 similarity'),
 (np.float64(0.012130990069664978), 'geo__Cluster 38 similarity'),
 (np.float64(0.012066994025263119), 'geo__Cluster 7 similarity'),
 (np.float64(0.012064355042782849), 'geo__Cluster 31 similarity'),
 (np.float64(0.011671789025244357), 'geo__Cluster 14 similarity'),
 (np.float64(0.011163069387541882), 'geo__Cluster 34 similarity'),
 (np.float64(0.01114723938638501), 'geo__Cluster 30 similarity'),
 (np.float64(0.01056669467923706), 'geo__Cluster 42 similarity'),
 (np.float64(0.010465541604367913),
  'remainder__housing_median_age'),
 (np.float64(0.010417127807391324), 'geo__Cluster 0 similarity'),
 (np.float64(0.010252012560303626), 'geo__Cluster 4 similarity'),
 (np.float64(0.009927897156914035), 'geo__Cluster 1 similarity'),
 (np.float64(0.009460889696918749), 'geo__Cluster 37 similarity'),
 (np.float64(0.00920153229160331), 'geo__Cluster 44 similarity'),
 (np.float64(0.00909629080536353), 'geo__Cluster 28 similarity'),
 (np.float64(0.009003573908684077), 'geo__Cluster 8 similarity'),
 (np.float64(0.008344279448300396), 'geo__Cluster 11 similarity'),
 (np.float64(0.00825603703098938), 'geo__Cluster 25 similarity'),
 (np.float64(0.008154702099230438), 'geo__Cluster 15 similarity'),
 (np.float64(0.0074313465489394), 'geo__Cluster 36 similarity'),
 (np.float64(0.007286424154166043), 'geo__Cluster 33 similarity'),
 (np.float64(0.007079903258801585), 'geo__Cluster 23 similarity'),
 (np.float64(0.007034031486131196), 'geo__Cluster 27 similarity'),
 (np.float64(0.006974525375475104), 'log__total_rooms'),
 (np.float64(0.006859210733720631), 'geo__Cluster 16 similarity'),
 (np.float64(0.006689923833586895), 'log__population'),
 (np.float64(0.006447978357162211), 'geo__Cluster 20 similarity'),
 (np.float64(0.006145547907770251), 'geo__Cluster 24 similarity'),
 (np.float64(0.006105563765401244), 'log__total_bedrooms'),
 (np.float64(0.00605237765053955), 'geo__Cluster 19 similarity'),
 (np.float64(0.005971436006316572), 'geo__Cluster 13 similarity'),
 (np.float64(0.005957493296827358), 'log__households'),
```

```
 (np.float64(0.004860197454898535), 'geo__Cluster 9 similarity'),
 (np.float64(0.004625832242554027), 'geo__Cluster 12 similarity'),
 (np.float64(0.004419952746718312), 'geo__Cluster 5 similarity'),
 (np.float64(0.0035759270467356683), 'geo__Cluster 29
similarity'),
 (np.float64(0.0025604887780587902), 'cat__ocean_proximity_<1H
OCEAN'),
 (np.float64(0.0015356253939294946), 'cat__ocean_proximity_NEAR
OCEAN'),
 (np.float64(0.000515415633411439), 'cat__ocean_proximity_NEAR
BAY'),
 (np.float64(1.467861268817004e-05),
'cat__ocean_proximity_ISLAND')]
```

## Avalie seu Sistema no Conjunto de Teste

In [170]:
```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = final_model.predict(X_test)

final_rmse = root_mean_squared_error(y_test, final_predictions)
print(final_rmse)
```

```
41556.053474456494
```

Podemos calcular um intervalo de confiança de 95% para o RMSE no conjunto de teste:

In [171]:
```
from scipy import stats

def rmse(squared_errors):
    return np.sqrt(np.mean(squared_errors))

confidence = 0.95
squared_errors = (final_predictions - y_test) ** 2
boot_result = stats.bootstrap([squared_errors], rmse,
                              confidence_level=confidence,
random_state=42)
rmse_lower, rmse_upper = boot_result.confidence_interval
```

In [172]:
```
rmse_lower, rmse_upper
```

Out[172]: (np.float64(39629.60159569332), np.float64(43816.66777858479))

# Persistencia do Modelo usando joblib

Salvando o modelo final:

In [ ]:
```
import joblib

joblib.dump(final_model, "my_california_housing_model.pkl")
```

```
['my_california_housing_model.pkl']
```

Agora você pode fazer o deploy do modelo em Produção. Por exemplo, o código abaixo pode ser um scrpit executando em Produção.

In [ ]:
```
import joblib

# extra code – excluded for conciseness
from sklearn.cluster import KMeans
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.metrics.pairwise import rbf_kernel

def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

#class ClusterSimilarity(BaseEstimator, TransformerMixin):
#    [...]

final_model_reloaded = joblib.load("my_california_housing_model.pkl")

new_data = housing.iloc[:5]  # pretend these are new districts
predictions = final_model_reloaded.predict(new_data)
```

In [ ]:
```
predictions
```

```
array([442520.14, 452569.09, 105878.  ,  99171.  , 330284.02])
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.